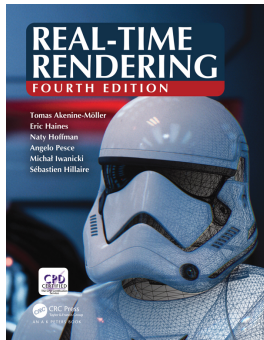


Shading and Texturing

Introduction to Computer Graphics - Lecture 05

Pirmin Pfeifer



Real-Time Rendering,
Fourth Edition
T. Akenine-Möller Et AL.

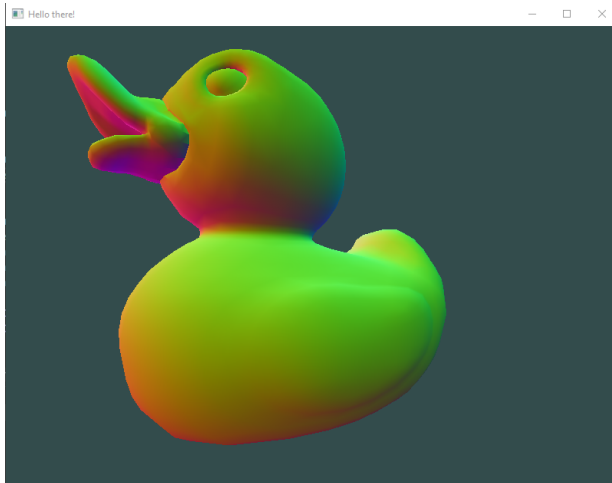
[View Online](#)

Related Chapters:

- ▶ 5 Basic Shading
- ▶ 6 Texturing

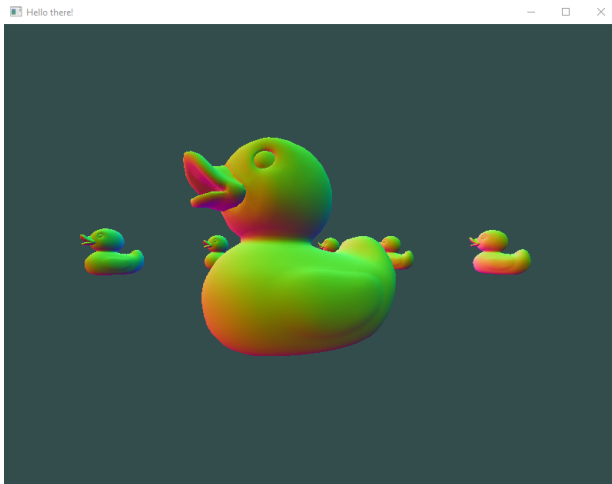
Shapes

We already learned how to render an shape:



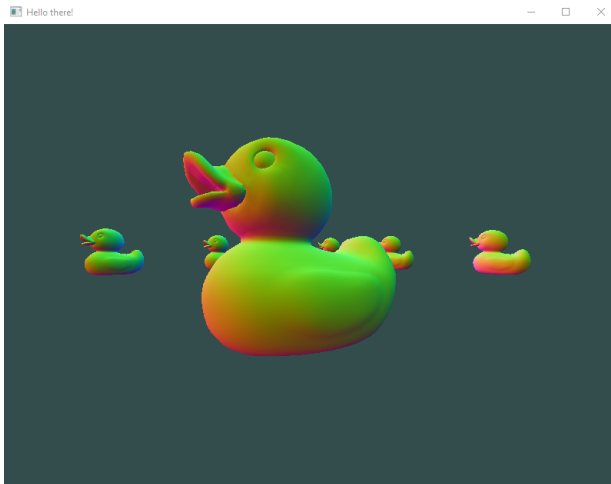
Shapes and Scenes

We already learned how to render a shape and create a scene:



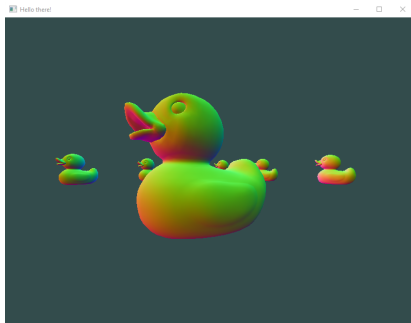
Shapes and Scenes

We already learned how to render a shape and create a scene:



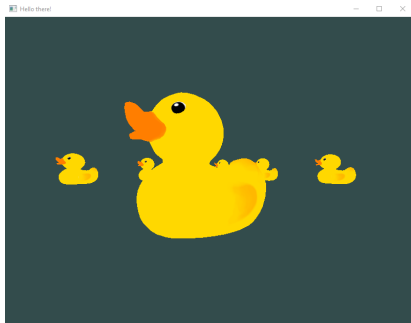
But the duck should also look like a duck and not just be shaped like it!

Shading and Texturing



To make the duck look more realistic
we need to add:

Shading and Texturing

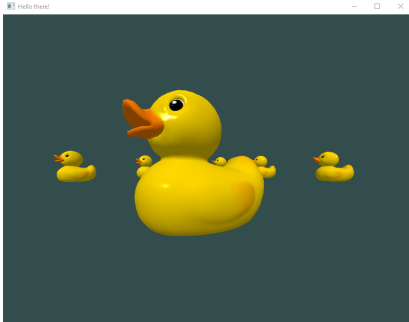


To make the duck look more realistic we need to add:

- Colors (Texture)



Shading and Texturing

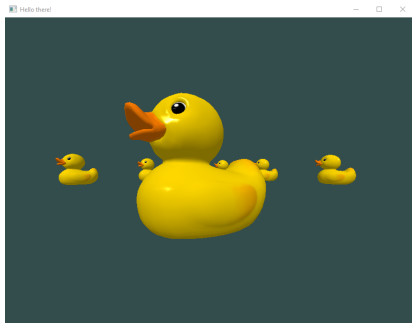


To make the duck look more realistic we need to add:

- ▶ Colors (Texture)
- ▶ Lighting (Local Phong Model)



Shading and Texturing



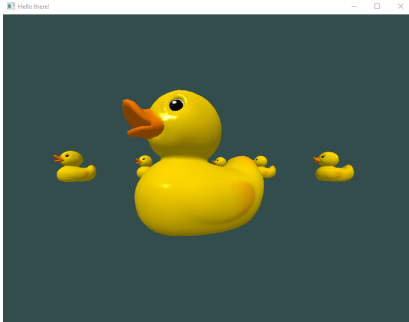
To achieve this we use:

To make the duck look more realistic we need to add:

- ▶ Colors (Texture)
- ▶ Lighting (Local Phong Model)



Shading and Texturing



To achieve this we use:

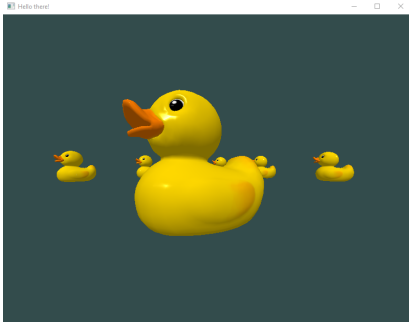
- ▶ Texture Coordinates

To make the duck look more realistic we need to add:

- ▶ Colors (Texture)
- ▶ Lighting (Local Phong Model)



Shading and Texturing



To achieve this we use:

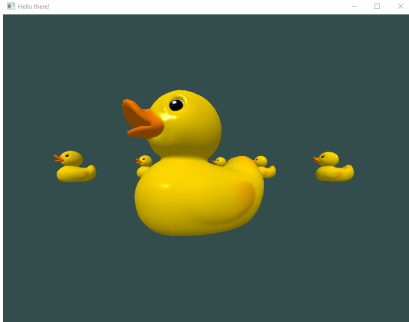
- ▶ Texture Coordinates
- ▶ Texture Samplers

To make the duck look more realistic we need to add:

- ▶ Colors (Texture)
- ▶ Lighting (Local Phong Model)



Shading and Texturing



To make the duck look more realistic we need to add:

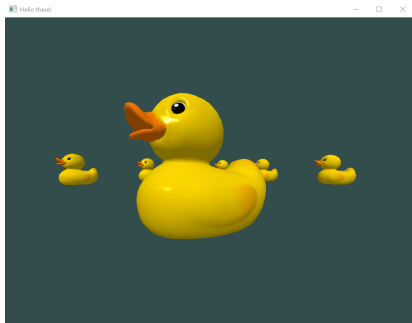
- ▶ Colors (Texture)
- ▶ Lighting (Local Phong Model)

To achieve this we use:

- ▶ Texture Coordinates
- ▶ Texture Samplers
- ▶ Image Data



Shading and Texturing



To make the duck look more realistic we need to add:

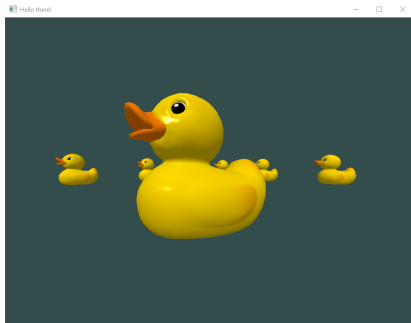
- ▶ Colors (Texture)
- ▶ Lighting (Local Phong Model)

To achieve this we use:

- ▶ Texture Coordinates
- ▶ Texture Samplers
- ▶ Image Data
- ▶ Normal Vectors



Shading and Texturing



To make the duck look more realistic we need to add:

- ▶ Colors (Texture)
- ▶ Lighting (Local Phong Model)

To achieve this we use:

- ▶ Texture Coordinates
- ▶ Texture Samplers
- ▶ Image Data
- ▶ Normal Vectors
- ▶ and more!



Shading Model

Shading Model

A description of how the color of an object is determined, based on different factors, such as surface orientation, view direction, and lighting.

Shading Model

A description of how the color of an object is determined, based on different factors, such as surface orientation, view direction, and lighting.

Shading models can be split into different categories:

Shading Model

A description of how the color of an object is determined, based on different factors, such as surface orientation, view direction, and lighting.

Shading models can be split into different categories:

Local Shading

or

Global Shading

Shading Model

A description of how the color of an object is determined, based on different factors, such as surface orientation, view direction, and lighting.

Shading models can be split into different categories:

Local Shading

or

Global Shading

Photorealistic

or

Stylized

Gooch Shading

Currently our duck is shaded 100% flat¹:

Currently our duck is shaded 100% flat¹:



¹Each pixel covered by the duck has the same color. Which is also called uniform flat shading.

Gooch Shading

A simple idea to add lighting:

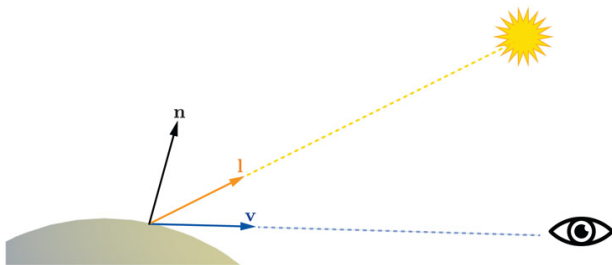
A simple idea to add lighting:

Use the *dot product* between the *normal* and the *light direction*!

Gooch Shading

A simple idea to add lighting:

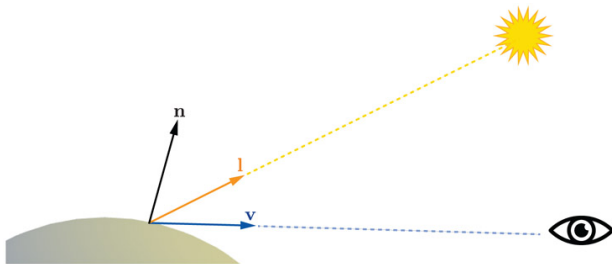
Use the *dot product* between the *normal* and the *light direction*!



Gooch Shading

A simple idea to add lighting:

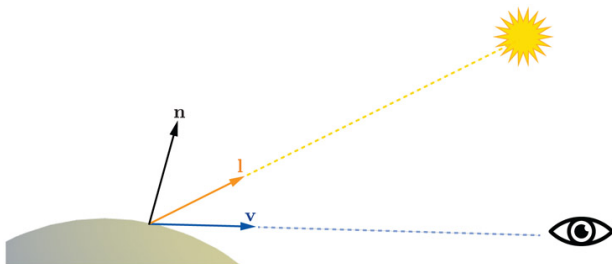
Use the *dot product* between the *normal* and the *light direction*!



When both direction are normalized the dot product has results in:

A simple idea to add lighting:

Use the *dot product* between the *normal* and the *light direction*!

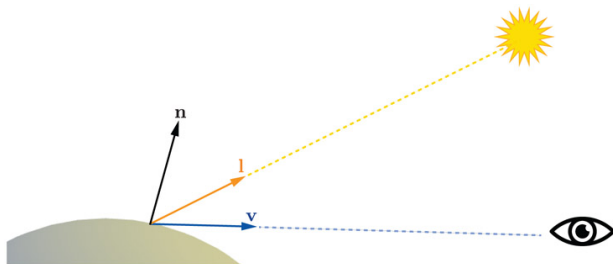


When both direction are normalized the dot product has results in:

- **1**: normal equals light direction

A simple idea to add lighting:

Use the *dot product* between the *normal* and the *light direction*!

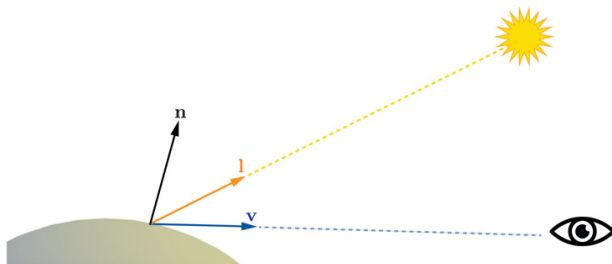


When both direction are normalized the dot product has results in:

- ▶ **1**: normal equals light direction
- ▶ **0**: normal and light direction are orthogonal

A simple idea to add lighting:

Use the *dot product* between the *normal* and the *light direction*!



When both direction are normalized the dot product has results in:

- ▶ **1**: normal equals light direction
- ▶ **0**: normal and light direction are orthogonal
- ▶ **-1**: normal and light direction oppose each other

Gooch Shading

Therefore we can get a value t between **0** and **1**
from the *normal* n and the *light direction* l :

Gooch Shading

Therefore we can get a value t between **0** and **1** from the *normal* n and the *light direction* l :

$$t = (n \cdot l) \cdot 0.5 + 0.5$$

Gooch Shading

Therefore we can get a value t between **0** and **1** from the *normal* n and the *light direction* l :

$$t = (n \cdot l) \cdot 0.5 + 0.5$$

Which we can use to blend between a warm and cold color:

Gooch Shading

Therefore we can get a value t between **0** and **1** from the *normal* n and the *light direction* l :

$$t = (n \cdot l) \cdot 0.5 + 0.5$$

Which we can use to blend between a warm and cold color:

$$color = t \cdot c_{warm} + (1 - t) \cdot c_{cold}$$

Therefore we can get a value t between **0** and **1** from the *normal* n and the *light direction* l :

$$t = (n \cdot l) \cdot 0.5 + 0.5$$

Which we can use to blend between a warm and cold color:

$$color = t \cdot c_{warm} + (1 - t) \cdot c_{cold}$$

This color is calculated per pixel on our duck:

Gooch Shading

Therefore we can get a value t between **0** and **1** from the *normal* n and the *light direction* l :

$$t = (n \cdot l) \cdot 0.5 + 0.5$$

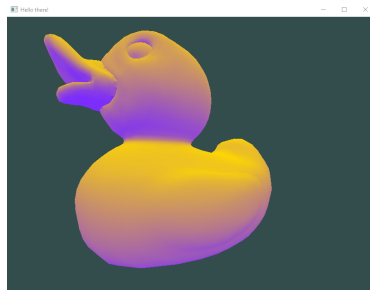
Which we can use to blend between a warm and cold color:

$$color = t \cdot c_{warm} + (1 - t) \cdot c_{cold}$$

This color is calculated per pixel on our duck:



Uniform flat shading



Gooch shading

Light and its interaction with the surface of the model is the primary driver for our shading.

Light and its interaction with the surface of the model is the primary driver for our shading.

Light sources can have many different attributes:

Light and its interaction with the surface of the model is the primary driver for our shading.

Light sources can have many different attributes:

- Size

Light and its interaction with the surface of the model is the primary driver for our shading.

Light sources can have many different attributes:

- ▶ Size
- ▶ Shape

Light and its interaction with the surface of the model is the primary driver for our shading.

Light sources can have many different attributes:

- ▶ Size
- ▶ Shape
- ▶ Color

Light and its interaction with the surface of the model is the primary driver for our shading.

Light sources can have many different attributes:

- ▶ Size
- ▶ Shape
- ▶ Color
- ▶ Intensity

Light and its interaction with the surface of the model is the primary driver for our shading.

Light sources can have many different attributes:

- ▶ Size
- ▶ Shape
- ▶ Color
- ▶ Intensity
- ▶ Direction

Light and its interaction with the surface of the model is the primary driver for our shading.

Light sources can have many different attributes:

- ▶ Size
- ▶ Shape
- ▶ Color
- ▶ Intensity
- ▶ Direction

The simple Gooch shading just used a simple light direction.

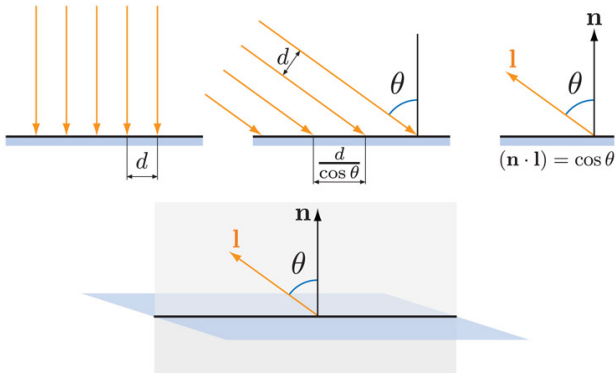
Light and its interaction with the surface of the model is the primary driver for our shading.

Light sources can have many different attributes:

- ▶ Size
- ▶ Shape
- ▶ Color
- ▶ Intensity
- ▶ Direction

The simple Gooch shading just used a simple light direction. Photorealistic lighting needs to consider all attributes.

Light Surface Interaction



We can check if the light is hitting a object by evaluating:

$$0 \geq n \cdot l$$

We can check if the light is hitting a object by evaluating:

$$0 \geq n \cdot l$$

Which can be extended to a gradient k ($0 := no\ light$; $1 := full\ light$):

$$k = \max(n \cdot l, 0)$$

We can check if the light is hitting a object by evaluating:

$$0 \geq n \cdot l$$

Which can be extended to a gradient k ($0 := no\ light$; $1 := full\ light$):

$$k = \max(n \cdot l, 0)$$

Combine with color to create simple lit shading:

$$c_{shaded} = c_{unlit} + k \cdot c_{lit}$$

We can check if the light is hitting a object by evaluating:

$$0 \geq n \cdot l$$

Which can be extended to a gradient k ($0 := no\ light$; $1 := full\ light$):

$$k = max(n \cdot l, 0)$$

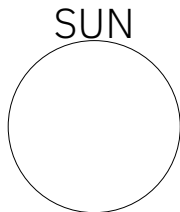
Combine with color to create simple lit shading:

$$c_{shaded} = c_{unlit} + k \cdot c_{lit}$$

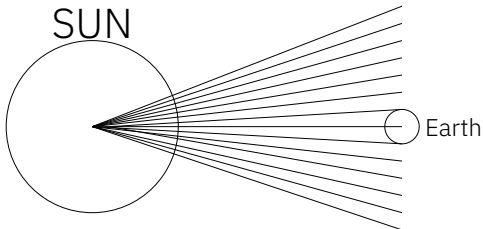
and with multiple (n) light sources:

$$c_{shaded} = c_{unlit} + c_{lit} \sum_{i=1}^n k_i$$

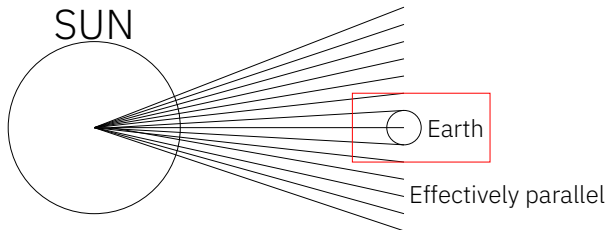
Directional Lights



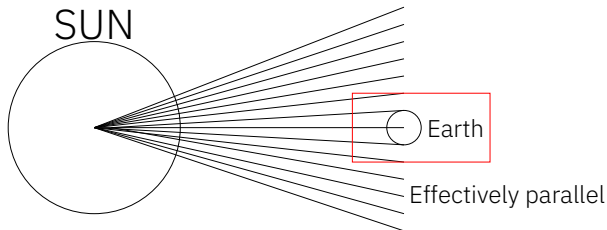
Directional Lights



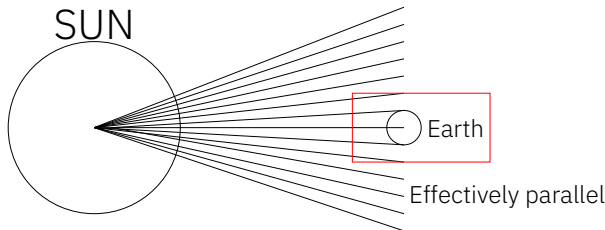
Directional Lights



Directional Lights

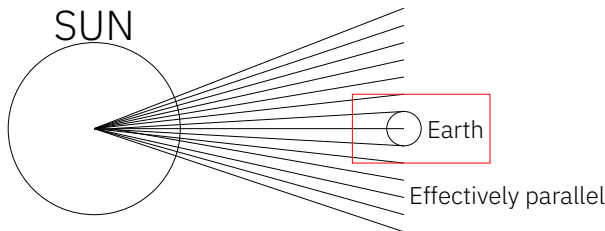


The rays of the sun effectively hit the earth in parallel.



The rays of the sun effectively hit the earth in parallel.
A *directional light* mimics this behavior and defines a global light direction.

Directional Lights



The rays of the sun effectively hit the earth in parallel.
A *directional light* mimics this behavior and defines a global light direction.
It can also define a color and Intensity.

A *point light* defines a *point* p_l in space from which the light is emitted.

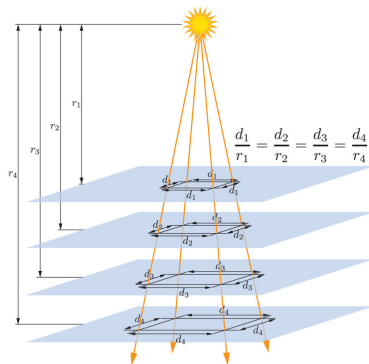
A *point light* defines a *point* p_l in space from which the light is emitted.
The light direction l at a point p_m is
relative to p_l

A *point light* defines a *point* p_l in space from which the light is emitted. The light direction l at a point p_m is relative to p_l

$$l = \frac{p_l - p_m}{\|p_l - p_m\|}$$

A *point light* defines a *point* p_l in space from which the light is emitted. The light direction l at a point p_m is relative to p_l

$$l = \frac{p_l - p_m}{\|p_l - p_m\|}$$

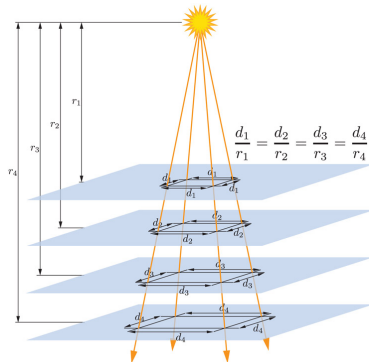


Point Lights

A *point light* defines a *point* p_l in space from which the light is emitted.
The light direction l at a point p_m is
relative to p_l

$$l = \frac{p_l - p_m}{\|p_l - p_m\|}$$

We want our point light to loose
Intensity over distance:



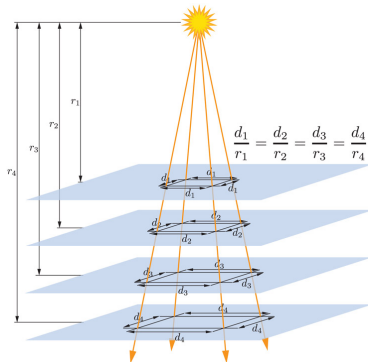
Point Lights

A *point light* defines a *point* p_l in space from which the light is emitted.
The light direction l at a point p_m is
relative to p_l

$$l = \frac{p_l - p_m}{\|p_l - p_m\|}$$

We want our point light to loose
Intensity over distance:

$$d = p_l - p_m$$



Point Lights

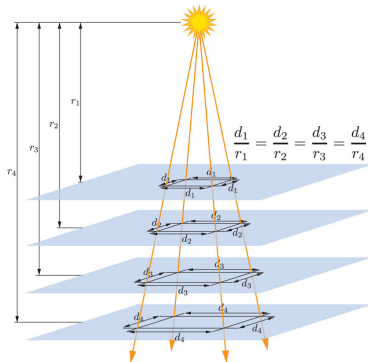
A *point light* defines a *point* p_l in space from which the light is emitted.
The light direction l at a point p_m is
relative to p_l

$$l = \frac{p_l - p_m}{\|p_l - p_m\|}$$

We want our point light to loose
Intensity over distance:

$$d = p_l - p_m$$

$$r = \sqrt{\|d\|}$$



Point Lights

A *point light* defines a *point* p_l in space from which the light is emitted.
The light direction l at a point p_m is
relative to p_l

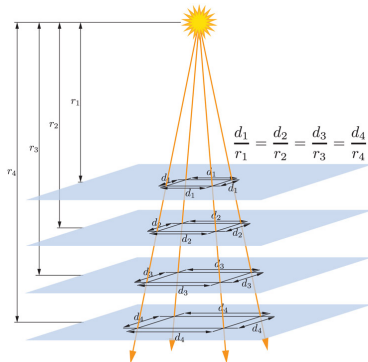
$$l = \frac{p_l - p_m}{\|p_l - p_m\|}$$

We want our point light to loose
Intensity over distance:

$$d = p_l - p_m$$

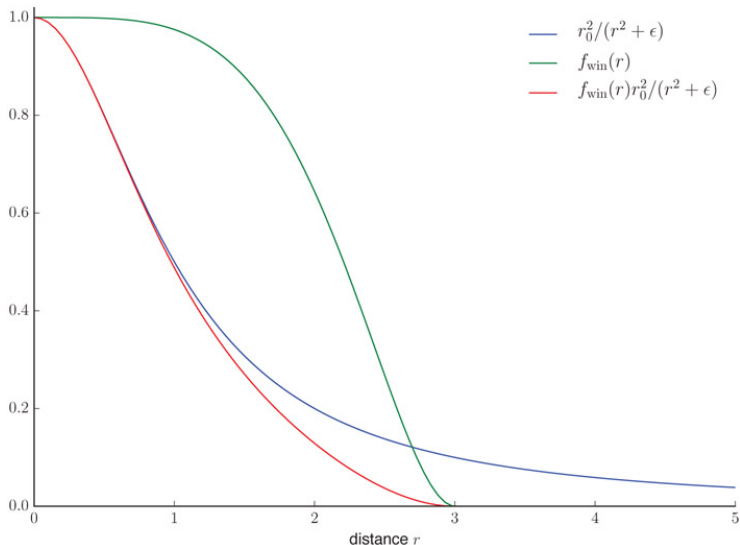
$$r = \sqrt{\|d\|}$$

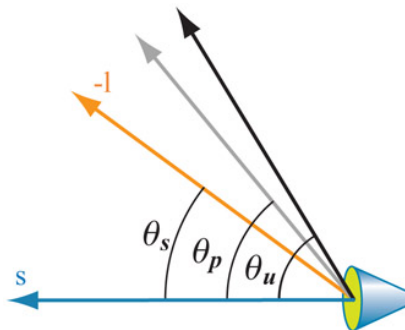
$$l = \frac{d}{r}$$

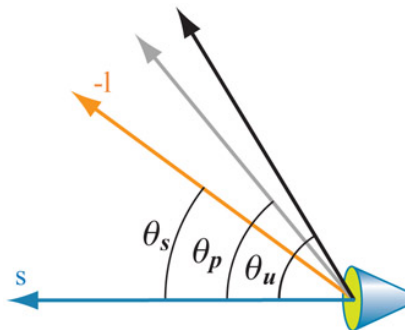


Point Lights - Falloff

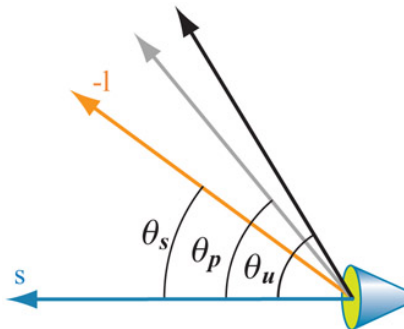
We can also use other functions for the Intensity Falloff:



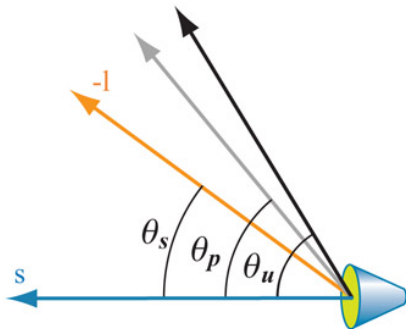




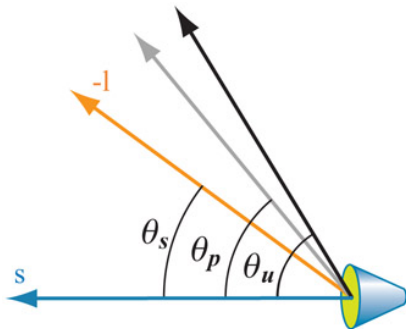
Spotlights have a position p , direction d and cone angle θ .



Spotlights have a position p , direction d and cone angle θ . Sometimes two angles are used to split between umbra θ_u and penumbra θ_p .

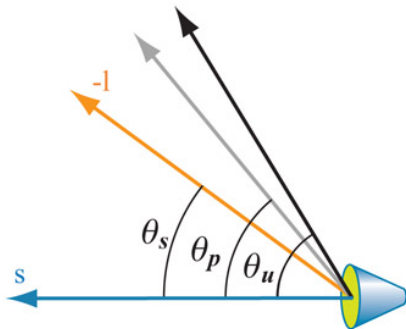


We can calculate a linear Falloff value t with:



We can calculate a linear Falloff value t with:

$$t_u = \frac{\cos(\theta_s), \cos(\theta_u)}{\cos(\theta_p), \cos(\theta_u)}$$

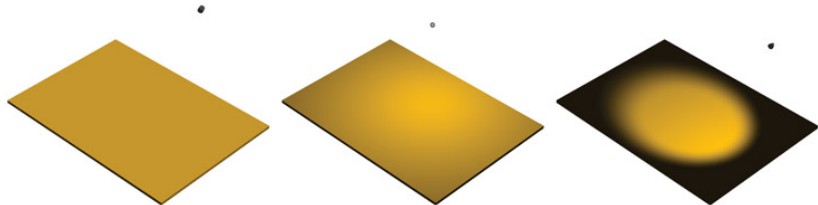


We can calculate a linear Falloff value t with:

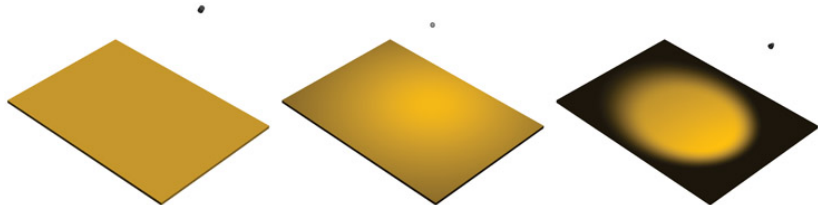
$$t_u = \frac{\cos(\theta_s), \cos(\theta_u)}{\cos(\theta_p), \cos(\theta_u)}$$

$$t = \text{clamp}(t_u, 0, 1)$$

Light Sources

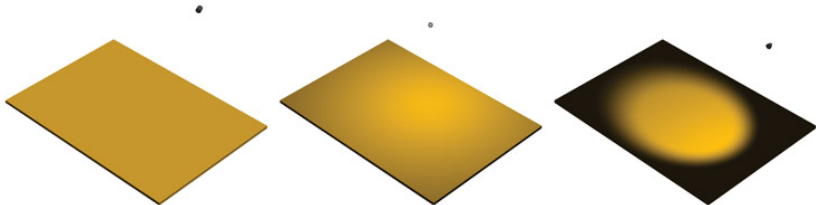


Light Sources



Directional Light

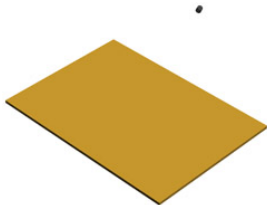
Light Sources



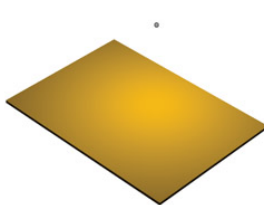
Directional Light

- one global light direction

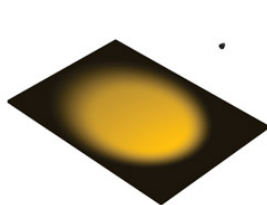
Light Sources



Directional Light

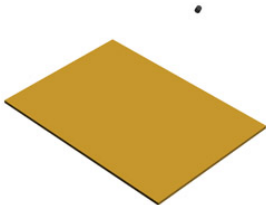


Point Light



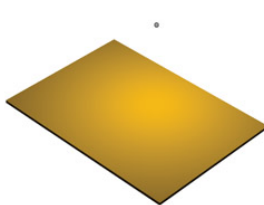
- one global light direction

Light Sources



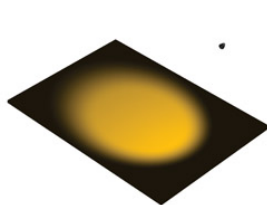
Directional Light

- one global light direction

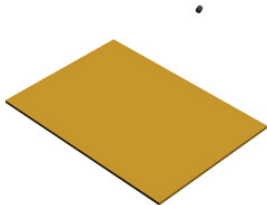


Point Light

- origin point

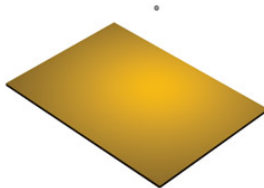


Light Sources



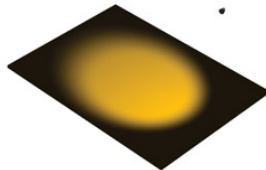
Directional Light

- ▶ one global light direction

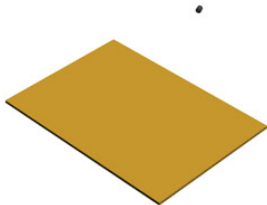


Point Light

- ▶ origin point
- ▶ shines radial in all directions

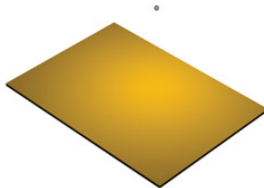


Light Sources



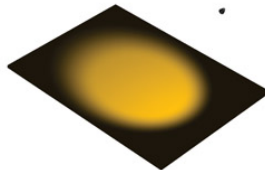
Directional Light

- ▶ one global light direction

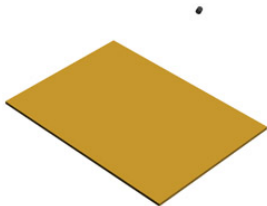


Point Light

- ▶ origin point
- ▶ shines radial in all directions
- ▶ falls off with distance

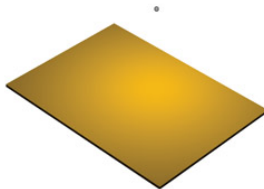


Light Sources



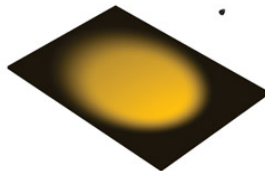
Directional Light

- ▶ one global light direction



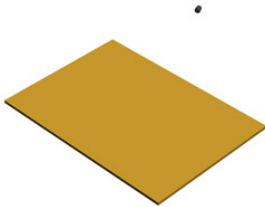
Point Light

- ▶ origin point
- ▶ shines radial in all directions
- ▶ falls off with distance



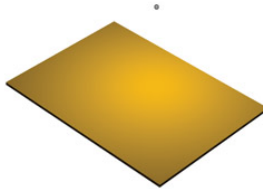
Spotlight

Light Sources



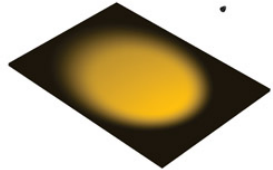
Directional Light

- ▶ one global light direction



Point Light

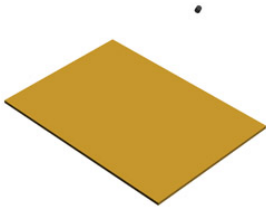
- ▶ origin point
- ▶ shines radial in all directions
- ▶ falls off with distance



Spotlight

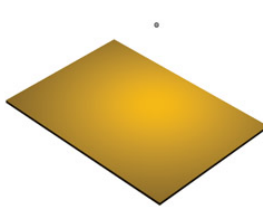
- ▶ origin point

Light Sources



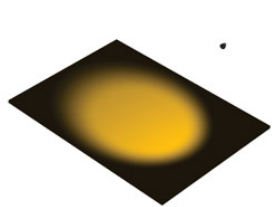
Directional Light

- ▶ one global light direction



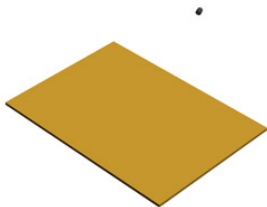
Point Light

- ▶ origin point
- ▶ shines radial in all directions
- ▶ falls off with distance



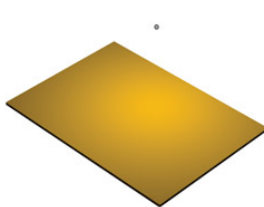
Spotlight

- ▶ origin point
- ▶ shines radial in a cone



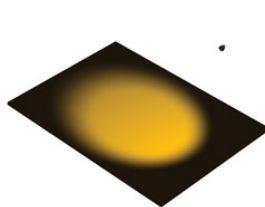
Directional Light

- ▶ one global light direction



Point Light

- ▶ origin point
- ▶ shines radial in all directions
- ▶ falls off with distance

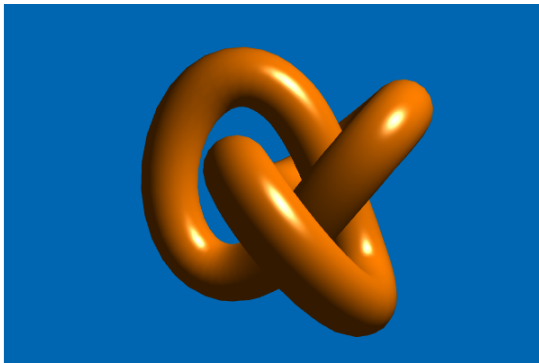


Spotlight

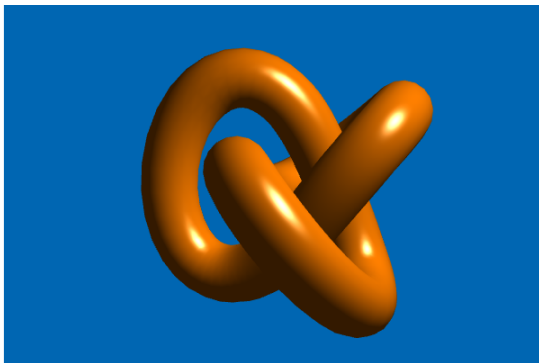
- ▶ origin point
- ▶ shines radial in a cone
- ▶ falls off with distance

Phong Reflection Shading incorporates more lighting terms to achieve local lighting:

Phong Reflection Shading incorporates more lighting terms to achieve local lighting:

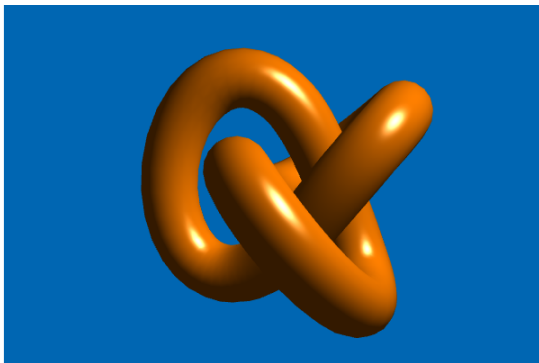


Phong Reflection Shading incorporates more lighting terms to achieve local lighting:



The light calculation is split into three parts:

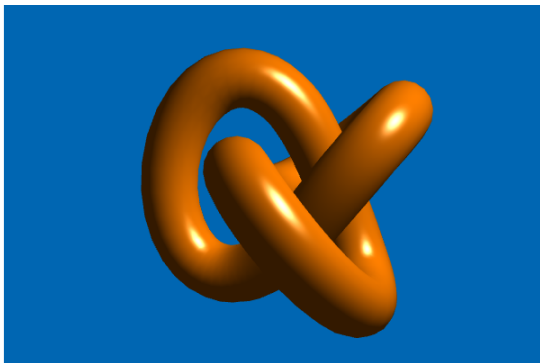
Phong Reflection Shading incorporates more lighting terms to achieve local lighting:



The light calculation is split into three parts:

- Ambient

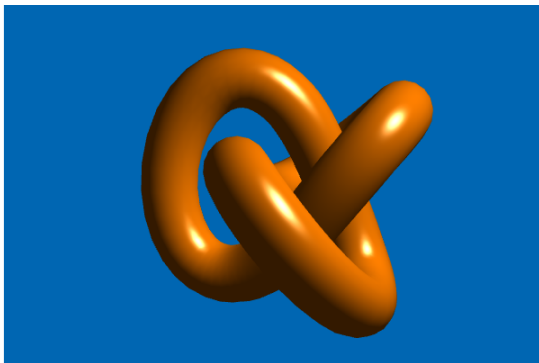
Phong Reflection Shading incorporates more lighting terms to achieve local lighting:



The light calculation is split into three parts:

- ▶ Ambient
- ▶ Diffuse

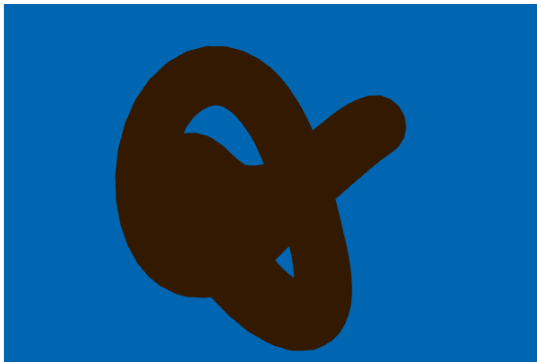
Phong Reflection Shading incorporates more lighting terms to achieve local lighting:



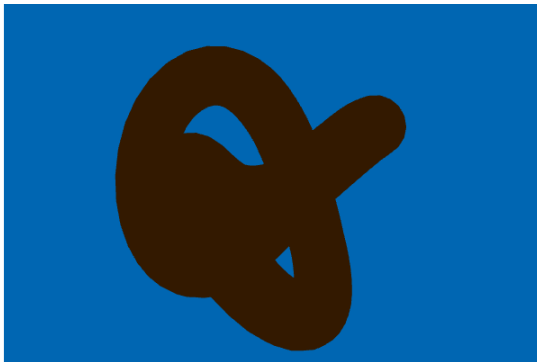
The light calculation is split into three parts:

- ▶ Ambient
- ▶ Diffuse
- ▶ Specular

Phong Shading - Ambient

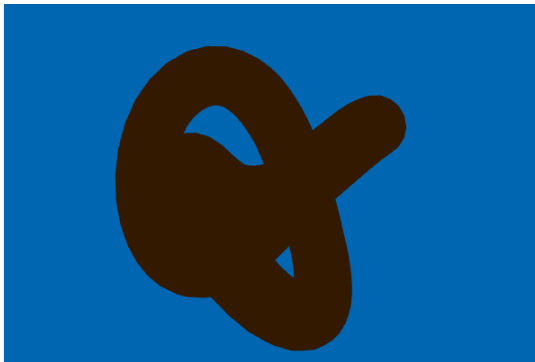


Phong Shading - Ambient



The ambient part is calculated from an ambient reflection k_a and an ambient color c_a :

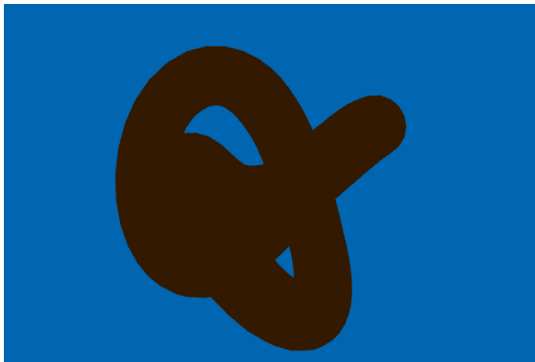
Phong Shading - Ambient



The ambient part is calculated from an ambient reflection k_a and an ambient color c_a :

$$ambient = k_a \cdot c_a$$

Phong Shading - Ambient

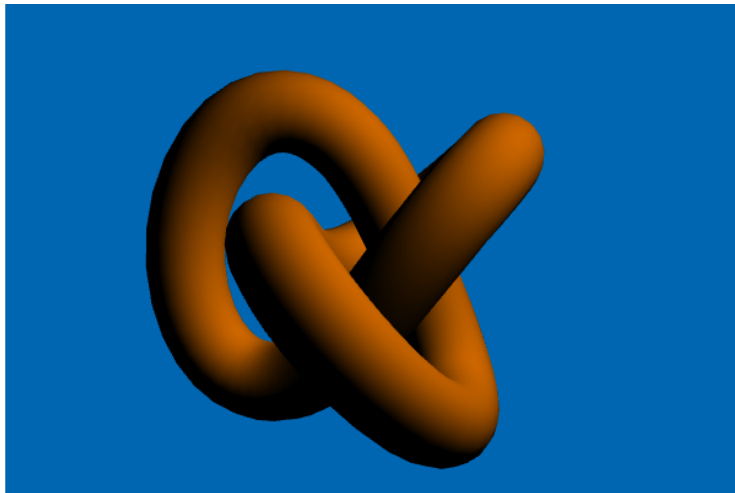


The ambient part is calculated from an ambient reflection k_a and an ambient color c_a :

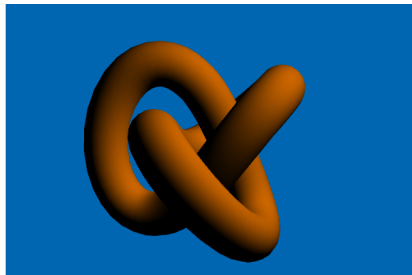
$$ambient = k_a \cdot c_a$$

which is just an flat shading controlled by the factor k_a .

Phong Shading - Diffuse

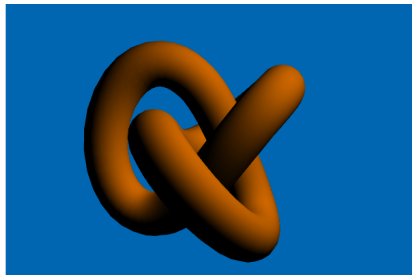


Phong Shading - Diffuse



The diffuse part is calculated from:

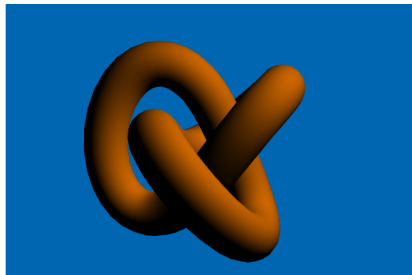
Phong Shading - Diffuse



The diffuse part is calculated from:

- $N :=$ surface normal

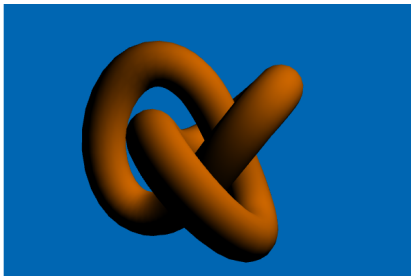
Phong Shading - Diffuse



The diffuse part is calculated from:

- ▶ N := surface normal
- ▶ L := light direction

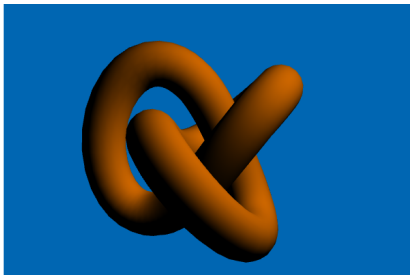
Phong Shading - Diffuse



The diffuse part is calculated from:

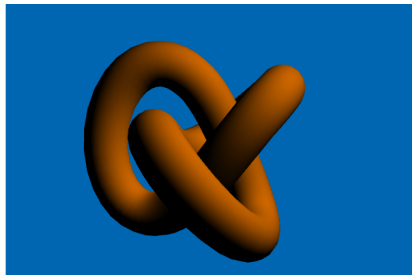
- ▶ N := surface normal
- ▶ L := light direction
- ▶ k_d := diffuse reflection

Phong Shading - Diffuse



The diffuse part is calculated from:

- ▶ N := surface normal
- ▶ L := light direction
- ▶ k_d := diffuse reflection
- ▶ c_d := diffuse color

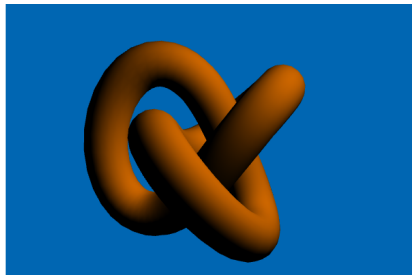


The diffuse part is calculated from:

- ▶ N := surface normal
- ▶ L := light direction
- ▶ k_d := diffuse reflection
- ▶ c_d := diffuse color

We calculate the *lambertian* using *Lambert's cosine law*:

$$lambertian = \max(N \cdot L, 0)$$



The diffuse part is calculated from:

- ▶ N := surface normal
- ▶ L := light direction
- ▶ k_d := diffuse reflection
- ▶ c_d := diffuse color

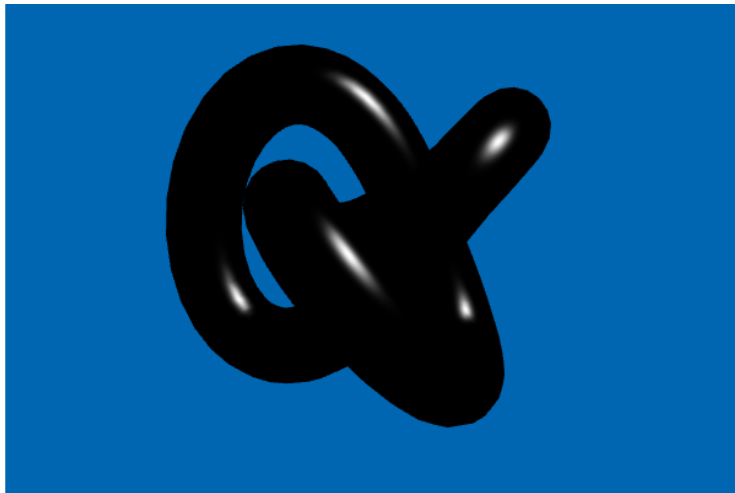
We calculate the *lambertian* using *Lambert's cosine law*:

$$lambertian = \max(N \cdot L, 0)$$

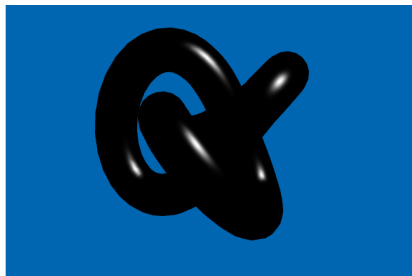
Then the diffuse part:

$$diffuse = k_d \cdot lambertian \cdot c_d$$

Phong Shading - Specular

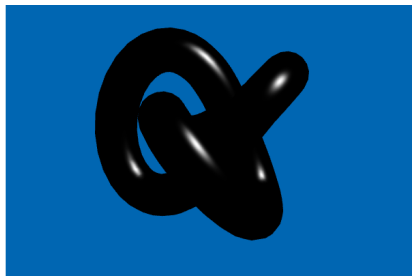


Phong Shading - Specular



The specular highlights are calculated from:

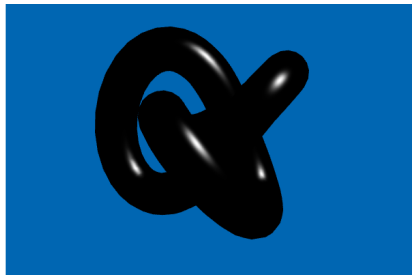
Phong Shading - Specular



The specular highlights are calculated from:

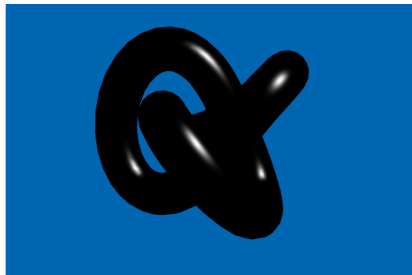
- ▶ $N :=$ surface normal

Phong Shading - Specular



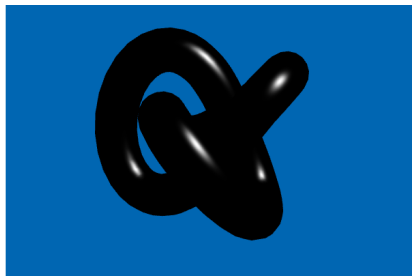
The specular highlights are calculated from:

- ▶ N := surface normal
- ▶ L := light direction
- ▶ V := direction to viewer



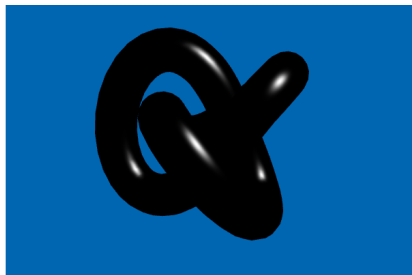
The specular highlights are calculated from:

- ▶ N := surface normal
- ▶ L := light direction
- ▶ V := direction to viewer
- ▶ k_s := specular reflection



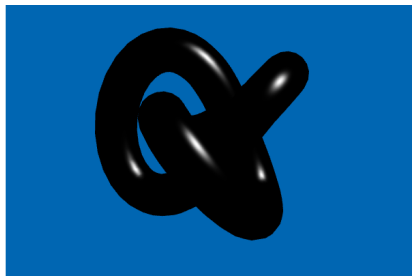
The specular highlights are calculated from:

- ▶ N := surface normal
- ▶ L := light direction
- ▶ V := direction to viewer
- ▶ k_s := specular reflection
- ▶ c_s := specular color



The specular highlights are calculated from:

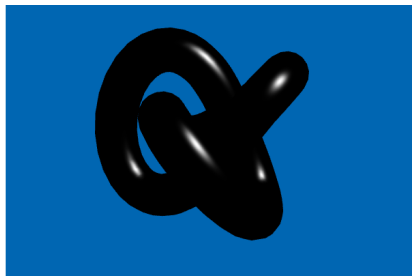
- ▶ N := surface normal
- ▶ L := light direction
- ▶ V := direction to viewer
- ▶ k_s := specular reflection
- ▶ c_s := specular color
- ▶ s := shininess value



First calculate R by reflecting L on N

The specular highlights are calculated from:

- ▶ N := surface normal
- ▶ L := light direction
- ▶ V := direction to viewer
- ▶ k_s := specular reflection
- ▶ c_s := specular color
- ▶ s := shininess value



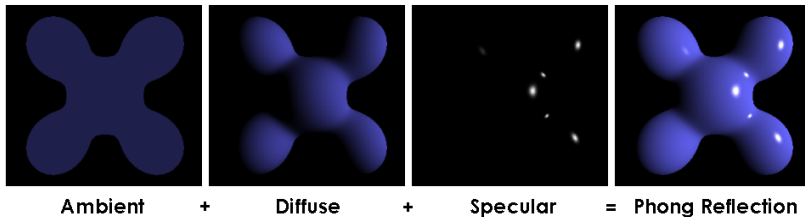
The specular highlights are calculated from:

- ▶ N := surface normal
- ▶ L := light direction
- ▶ V := direction to viewer
- ▶ k_s := specular reflection
- ▶ c_s := specular color
- ▶ s := shininess value

First calculate R by reflecting L on N and then the specular value:

$$specular = k_s \cdot \max(R \cdot V, 0)^s \cdot c_s$$

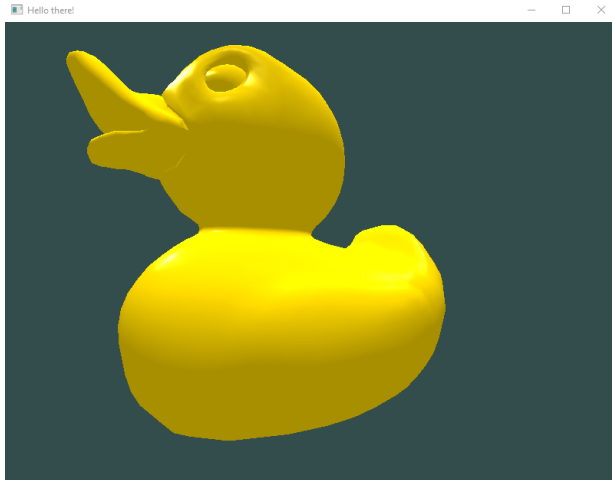
Phong Shading



An interactive demo is available online.

Phong Shading - Duck

Of course we can shade the duck with our new Phong shader:



We can finalize our lighting data at different points in our pipeline

We can finalize our lighting data at different points in our pipeline , changing appearance and cost of the calculations.

We can finalize our lighting data at different points in our pipeline , changing appearance and cost of the calculations.

- ▶ Vertex shader $\Rightarrow$ Evaluation per pre-tessellation vertex

We can finalize our lighting data at different points in our pipeline , changing appearance and cost of the calculations.

- ▶ Vertex shader $\Rightarrow$ Evaluation per pre-tessellation vertex
- ▶ Hull shader $\Rightarrow$ Evaluation per surface patch

We can finalize our lighting data at different points in our pipeline , changing appearance and cost of the calculations.

- ▶ Vertex shader $\Rightarrow$ Evaluation per pre-tessellation vertex
- ▶ Hull shader $\Rightarrow$ Evaluation per surface patch
- ▶ Domain shader $\Rightarrow$ Evaluation per post-tessellation vertex

We can finalize our lighting data at different points in our pipeline , changing appearance and cost of the calculations.

- ▶ Vertex shader ⇒ Evaluation per pre-tessellation vertex
- ▶ Hull shader ⇒ Evaluation per surface patch
- ▶ Domain shader ⇒ Evaluation per post-tessellation vertex
- ▶ Geometry shader ⇒ Evaluation per primitive

We can finalize our lighting data at different points in our pipeline , changing appearance and cost of the calculations.

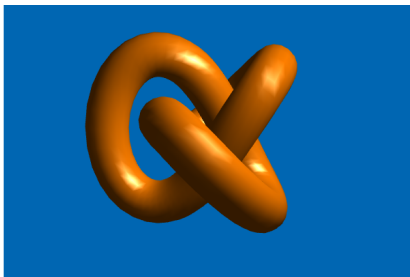
- ▶ Vertex shader $\Rightarrow$ Evaluation per pre-tessellation vertex
- ▶ Hull shader $\Rightarrow$ Evaluation per surface patch
- ▶ Domain shader $\Rightarrow$ Evaluation per post-tessellation vertex
- ▶ Geometry shader $\Rightarrow$ Evaluation per primitive
- ▶ Pixel shader $\Rightarrow$ Evaluation per pixel

Gouraud vs Phong

Gouraud and Phong Shading use the same equations at different points in the pipeline:

Gouraud vs Phong

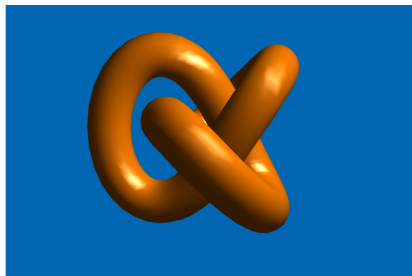
Gouraud and Phong Shading use the same equations at different points in the pipeline:



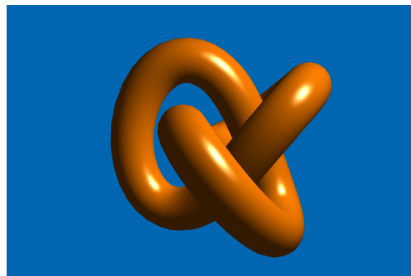
Per Vertex Evaluation

Gouraud vs Phong

Gouraud and Phong Shading use the same equations at different points in the pipeline:



Per Vertex Evaluation



Per Pixel Evaluation

Texture

A Texture describes the color, shape or profile of an objects surface.

Texture

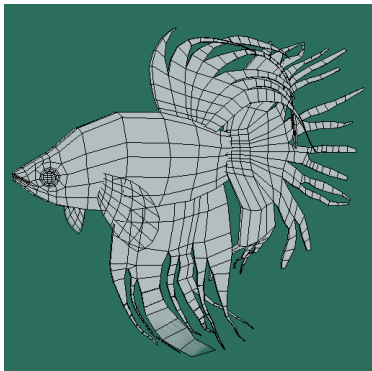
A Texture describes the color, shape or profile of an objects surface.

They are used to color the model, hide the lack of geometry and many more:

Texture

A Texture describes the color, shape or profile of an objects surface.

They are used to color the model, hide the lack of geometry and many more:

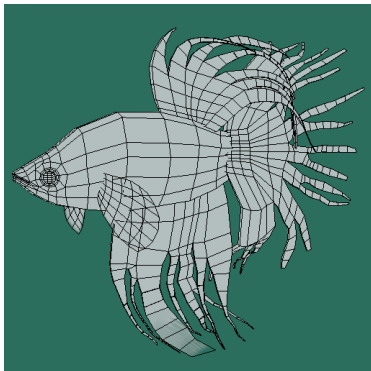


Plain Geometry

Texture

A Texture describes the color, shape or profile of an objects surface.

They are used to color the model, hide the lack of geometry and many more:



Plain Geometry



Geometry rendered with multiple textures

Texel

The pixels in the image texture are often called texels, to differentiate them from the pixels on the screen.

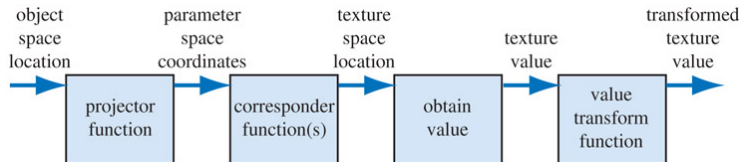
Texel

The pixels in the image texture are often called texels, to differentiate them from the pixels on the screen.

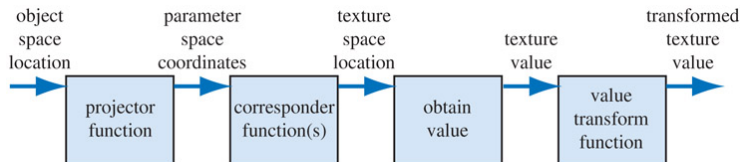
Texel Density

The number of texels mapped to one default square of a surface

The Texturing Pipeline



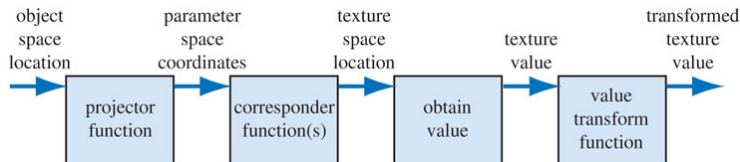
The Texturing Pipeline



- **Projector Function:** Map 3D point into 2D texture space²

²This process is called mapping

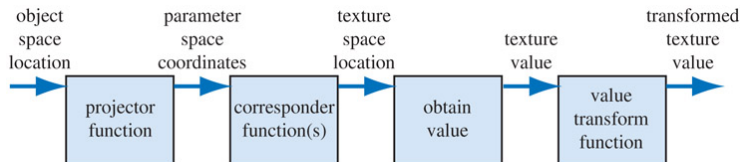
The Texturing Pipeline



- **Projector Function:** Map 3D point into 2D texture space²
- **Corosponder Function:** Handle coordinates outside of the texture (0,1)

²This process is called mapping

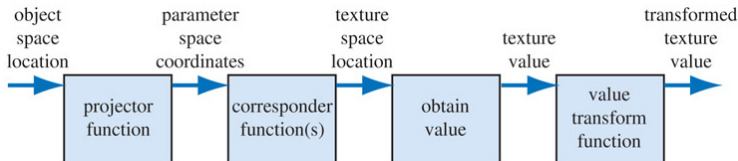
The Texturing Pipeline



- ▶ **Projector Function:** Map 3D point into 2D texture space²
- ▶ **Corosponder Function:** Handle coordinates outside of the texture (0,1)
- ▶ **Obtain Value:** Sample color of texture at coordinates

²This process is called mapping

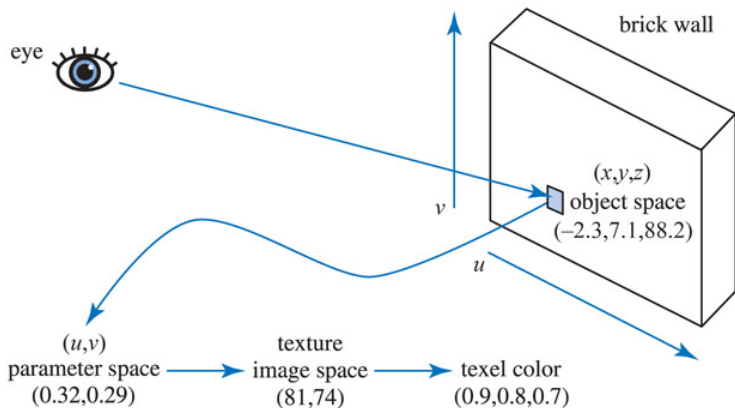
The Texturing Pipeline



- ▶ Projector Function: Map 3D point into 2D texture space²
- ▶ Corosponder Function: Handle coordinates outside of the texture (0,1)
- ▶ Obtain Value: Sample color of texture at coordinates
- ▶ Value Transform Function: Calculations done on the value by the shader

²This process is called mapping

Texturing Pipeline: Example



The Projector Function

The projector Function *unwraps* the 3D model into a 2D texture space:

Texturing

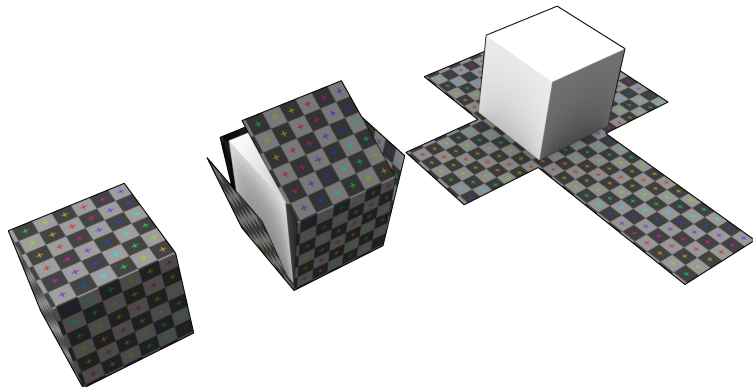


UV Image



The Projector Function

The projector Function *unwraps* the 3D model into a 2D texture space:



Three commonly used projection types are:

Three commonly used projection types are:

Spherical

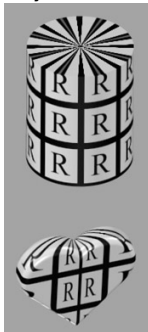


Three commonly used projection types are:

Spherical



Cylindrical

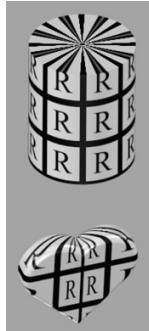


Three commonly used projection types are:

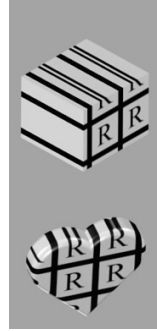
Spherical



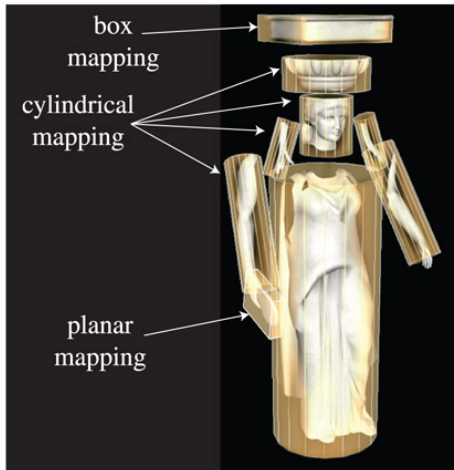
Cylindrical



Planar

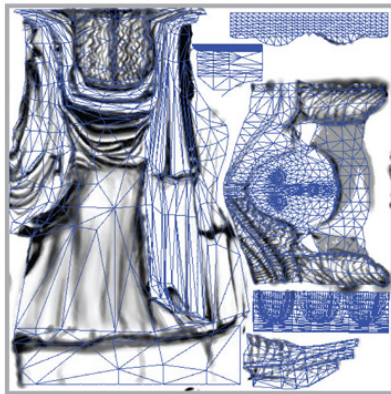


Projector Function: Example



In the modeling process a technical artist uses different projections and other techniques, to cut the 3D model open and create 2D coordinate for each vertex.

Projector Function: Example



In the modeling process a technical artist uses different projections and other techniques, to cut the 3D model open and create 2D coordinate for each vertex.

The Corresponder Function

The *corresponder Function* handles texture access outside of the range $[0,1]$.

The Corresponder Function

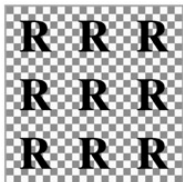
The *corresponder Function* handles texture access outside of the range $[0,1]$.

There are four corresponder Functions provided by the GPU:

The Corresponder Function

The *corresponder Function* handles texture access outside of the range $[0,1]$.

There are four corresponder Functions provided by the GPU:

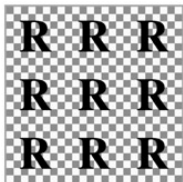


repeat

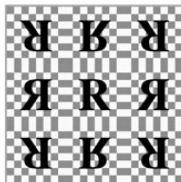
The Corresponder Function

The *corresponder Function* handles texture access outside of the range $[0,1]$.

There are four corresponder Functions provided by the GPU:



repeat

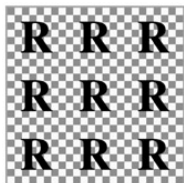


mirror

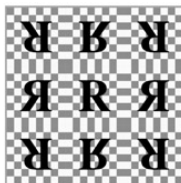
The Corresponder Function

The *corresponder Function* handles texture access outside of the range $[0,1]$.

There are four corresponder Functions provided by the GPU:



repeat



mirror

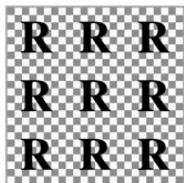


clamp to edge

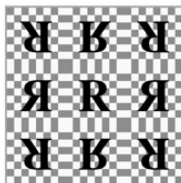
The Corresponder Function

The *corresponder Function* handles texture access outside of the range $[0,1]$.

There are four corresponder Functions provided by the GPU:



repeat



mirror



clamp to edge



border

After the correspondent functions are used to produce texture-space coordinates, the coordinates are used to obtain texture values.

After the correspondent functions are used to produce texture-space coordinates, the coordinates are used to obtain texture values.

This can either be done by:

After the correspondent functions are used to produce texture-space coordinates, the coordinates are used to obtain texture values.

This can either be done by:

- ▶ Reading the texel data from the corresponding image texture

After the correspondent functions are used to produce texture-space coordinates, the coordinates are used to obtain texture values.

This can either be done by:

- ▶ Reading the texel data from the corresponding image texture
- ▶ Executing a procedural texture function for the coordinates

The image texels are addressed in the shader with a 2D coordinate (uv) in the range of 0 to 1:

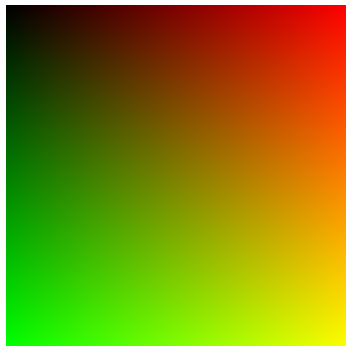


red: u-axis green: v-axis

The size of textures on the GPU are usually $2^n \times 2^m$, which are therefore called *power-of-two* textures:

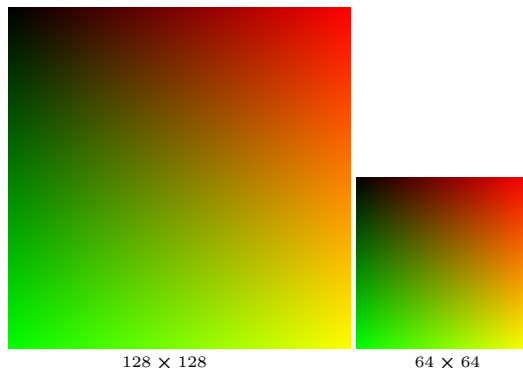
Image Texturing

The size of textures on the GPU are usually $2^n \times 2^m$, which are therefore called *power-of-two* textures:

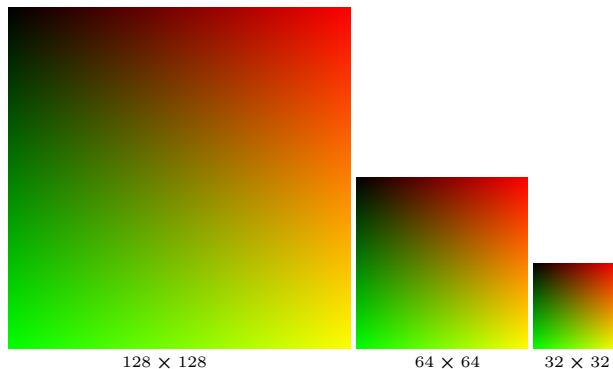


128 × 128

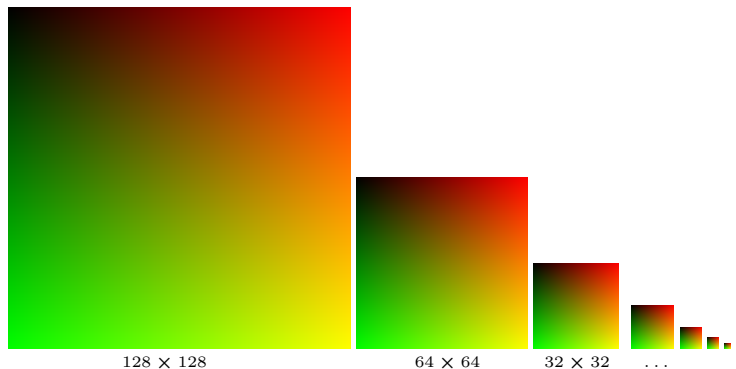
The size of textures on the GPU are usually $2^n \times 2^m$, which are therefore called *power-of-two* textures:



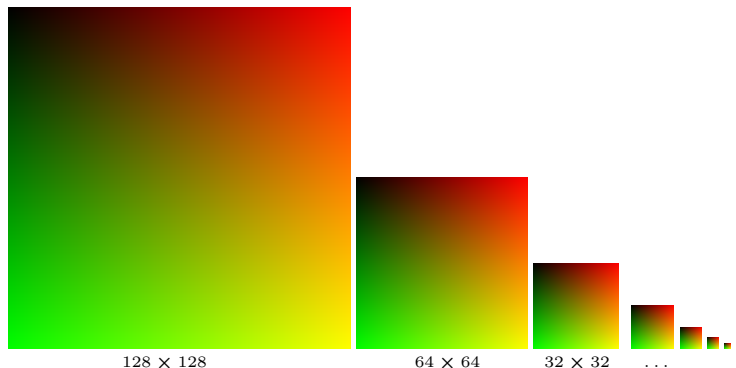
The size of textures on the GPU are usually $2^n \times 2^m$, which are therefore called *power-of-two* textures:



The size of textures on the GPU are usually $2^n \times 2^m$, which are therefore called *power-of-two* textures:



The size of textures on the GPU are usually $2^n \times 2^m$, which are therefore called *power-of-two* textures:



This enables easy and consistent generation of lower resolution versions of the texture. Additionally the mapping is more stable since textures can be expected to be quadratic.

Magnification

Some times a texture is rendered with more pixels than texels.

Magnification

Some times a texture is rendered with more pixels than texels.
Therefore a strategy of *Magnification* is needed for each texture.

Some times a texture is rendered with more pixels than texels.
Therefore a strategy of *Magnification* is needed for each texture.

There are three common strategies:

Magnification

Some times a texture is rendered with more pixels than texels.
Therefore a strategy of *Magnification* is needed for each texture.

There are three common strategies:



Nearest neighbor / Box
filtering

Color of nearest texel.

Magnification

Some times a texture is rendered with more pixels than texels.
Therefore a strategy of *Magnification* is needed for each texture.

There are three common strategies:



Nearest neighbor / Box
filtering

Color of nearest texel.



Bilinear filtering

Linear interpolate color of four
adjacent texel.

Magnification

Some times a texture is rendered with more pixels than texels.
Therefore a strategy of *Magnification* is needed for each texture.

There are three common strategies:



Nearest neighbor / Box
filtering

Color of nearest texel.



Bilinear filtering

Linear interpolate color of four
adjacent texel.

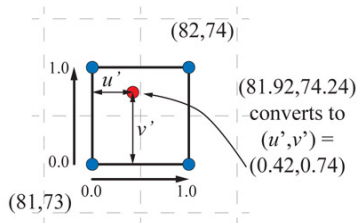
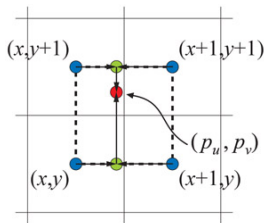


Cubic filtering

Weighted average of the nearest
 5×5 texels.

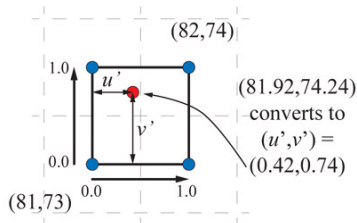
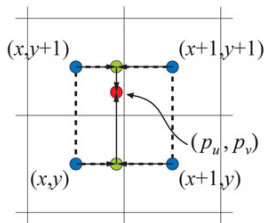
Magnification - Bilinear Filter

For Bilinear Filtering first the four adjacent points are calculated:



Magnification - Bilinear Filter

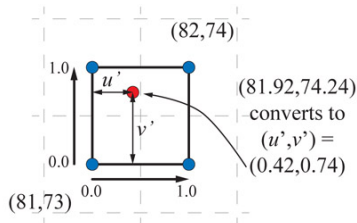
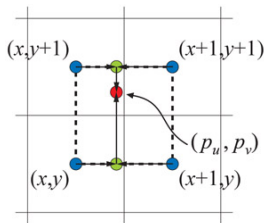
For Bilinear Filtering first the four adjacent points are calculated:



Additionally (u', v') are calculated.

Magnification - Bilinear Filter

For Bilinear Filtering first the four adjacent points are calculated:

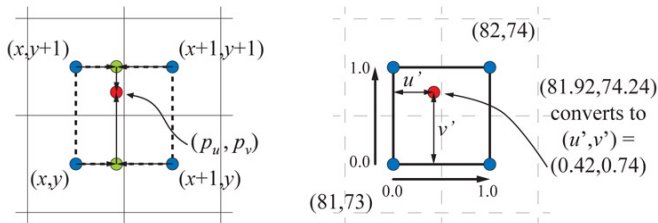


Additionally (u', v') are calculated. The pixel color is interpolated:

$$b(p_u, p_v) =$$

Magnification - Bilinear Filter

For Bilinear Filtering first the four adjacent points are calculated:



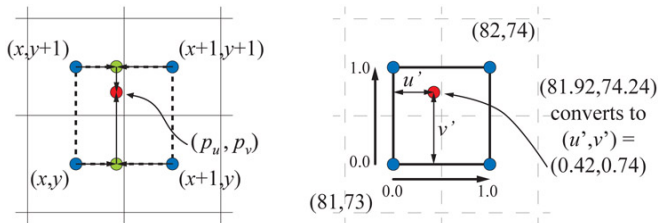
Additionally (u', v') are calculated. The pixel color is interpolated:

$$b(p_u, p_v) = (1 - v')((1 - u')t(x, y) + u't(x + 1, y))$$

Note: $t(x, y)$ = texture read at texel x, y

Magnification - Bilinear Filter

For Bilinear Filtering first the four adjacent points are calculated:



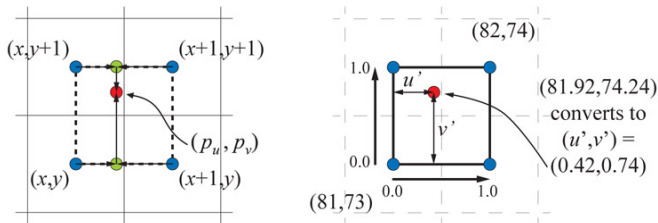
Additionally (u', v') are calculated. The pixel color is interpolated:

$$b(p_u, p_v) = (1 - v')((1 - u')t(x, y) + u't(x + 1, y)) \\ + v'((1 - u')t(x, y + 1) + u't(x + 1, y + 1))$$

Note: $t(x, y)$ = texture read at texel x, y

Magnification - Bilinear Filter

For Bilinear Filtering first the four adjacent points are calculated:



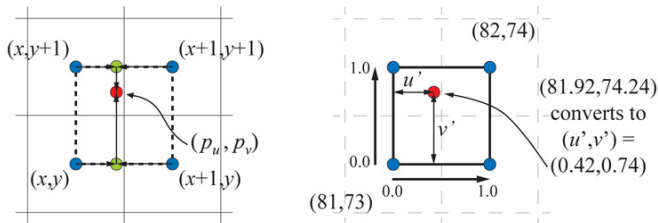
Additionally (u', v') are calculated. The pixel color is interpolated:

$$\begin{aligned} b(p_u, p_v) &= (1 - v')((1 - u')t(x, y) + u't(x + 1, y)) \\ &\quad + v'((1 - u')t(x, y + 1) + u't(x + 1, y + 1)) \\ &= (1 - u')(1 - v')t(x, y) + u'(1 - v')t(x + 1, y) \\ &\quad + v'(1 - u')t(x, y + 1) + v'u't(x + 1, y + 1) \end{aligned}$$

Note: $t(x, y)$ = texture read at texel x, y

Magnification - Bilinear Filter

For Bilinear Filtering first the four adjacent points are calculated:



Additionally (u', v') are calculated. The pixel color is interpolated:

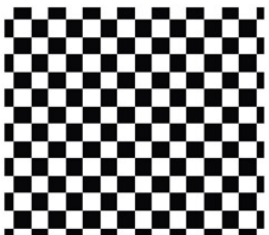
$$\begin{aligned} b(p_u, p_v) &= (1 - v')((1 - u')t(x, y) + u't(x + 1, y)) \\ &\quad + v'((1 - u')t(x, y + 1) + u't(x + 1, y + 1)) \\ &= (1 - u')(1 - v')t(x, y) + u'(1 - v')t(x + 1, y) \\ &\quad + v'(1 - u')t(x, y + 1) + v'u't(x + 1, y + 1) \end{aligned}$$

Note: $t(x, y)$ = texture read at texel x, y

Note: $(1 - u')(1 - v') + u'(1 - v') + v'(1 - u')v'u' + v'u' = 1$

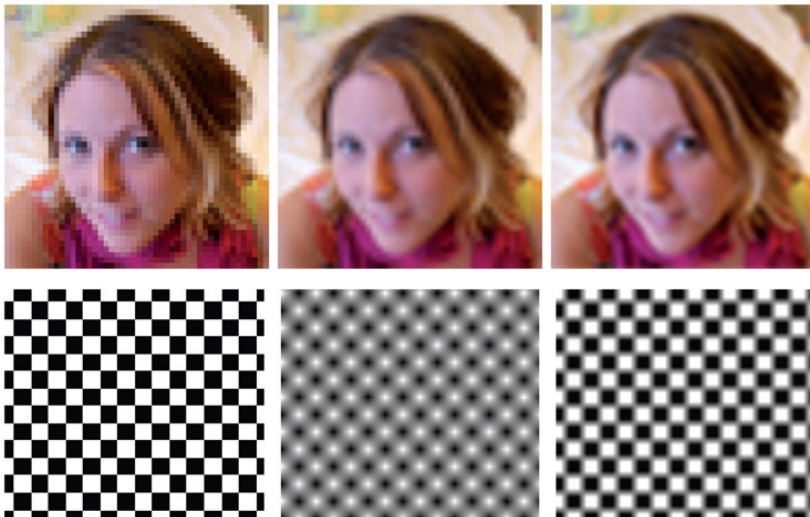
Magnification - Choosing a Filter

Depending on the content of the texture different filters are better:



Magnification - Choosing a Filter

Depending on the content of the texture different filters are better:



Bicubic strikes a balance between the other two approaches but is relatively expensive.

Magnification - Choosing a Filter

Since Bicubic is often too expensive, non linear interpolation function can be used to change the appearance of the bilinear approach:

Magnification - Choosing a Filter

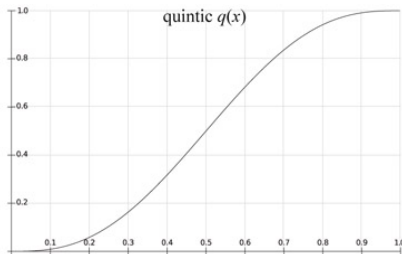
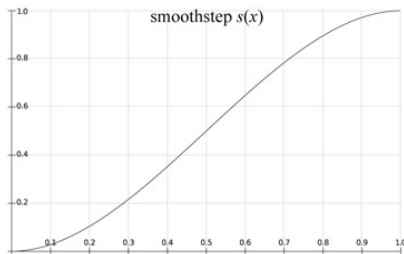
Since Bicubic is often too expensive, non linear interpolation function can be used to change the appearance of the bilinear approach:

$$\underbrace{s(x) = x^2(3 - 2x)}_{\text{smoothstep}} \quad \text{and} \quad \underbrace{q(x) = x^3(6x^2 - 15x + 10)}_{\text{quintic}}$$

Magnification - Choosing a Filter

Since Bicubic is often too expensive, non linear interpolation function can be used to change the appearance of the bilinear approach:

$$\underbrace{s(x) = x^2(3 - 2x)}_{\text{smoothstep}} \quad \text{and} \quad \underbrace{q(x) = x^3(6x^2 - 15x + 10)}_{\text{quintic}}$$

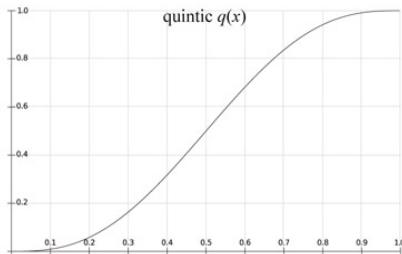
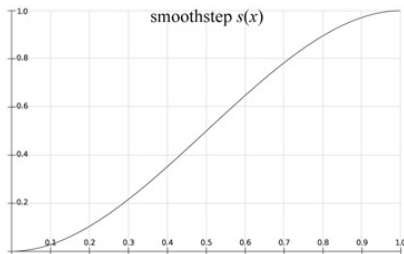


Both functions are zero for $x = 0$ and one for $x = 1$.

Magnification - Choosing a Filter

Since Bicubic is often too expensive, non linear interpolation function can be used to change the appearance of the bilinear approach:

$$\underbrace{s(x) = x^2(3 - 2x)}_{\text{smoothstep}} \quad \text{and} \quad \underbrace{q(x) = x^3(6x^2 - 15x + 10)}_{\text{quintic}}$$



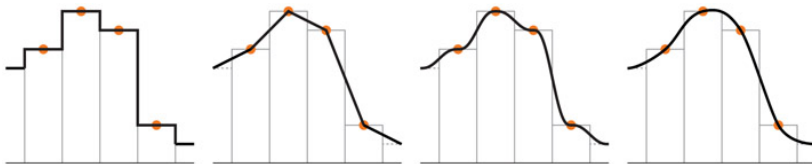
Both functions are zero for $x = 0$ and one for $x = 1$.
The derivative of both functions is zero both at $x = 0$ and $x = 1$.

Magnification - Choosing a Filter

Since Bicubic is often too expensive, non linear interpolation function can be used to change the appearance of the bilinear approach:

$$\underbrace{s(x) = x^2(3 - 2x)}_{\text{smoothstep}} \quad \text{and} \quad \underbrace{q(x) = x^3(6x^2 - 15x + 10)}_{\text{quintic}}$$

With the quintic we can mimic cubic interpolation with viewer texture reads:



When a texture is minimized, several texels may cover a pixels cell.

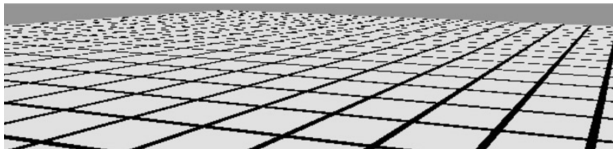
When a texture is minimized, several texels may cover a pixels cell.
Therefore a strategy of *Minification* is needed for each texture.

When a texture is minimized, several texels may cover a pixels cell.
Therefore a strategy of *Minification* is needed for each texture.

We can apply nearest neighbor also to Minification:

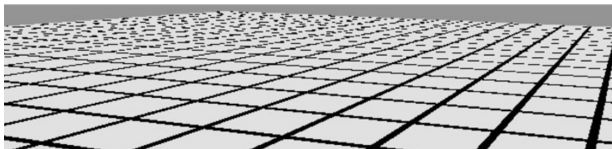
When a texture is minimized, several texels may cover a pixels cell. Therefore a strategy of *Minification* is needed for each texture.

We can apply nearest neighbor also to Minification:



When a texture is minimized, several texels may cover a pixels cell. Therefore a strategy of *Minification* is needed for each texture.

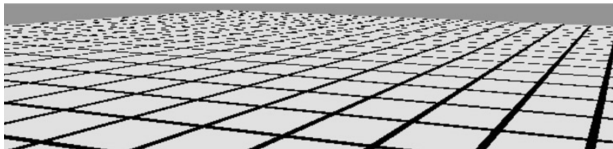
We can apply nearest neighbor also to Minification:



But this approach is prone to *aliasing*, which becomes even more prominent when the object is moving³.

When a texture is minimized, several texels may cover a pixels cell. Therefore a strategy of *Minification* is needed for each texture.

We can apply nearest neighbor also to Minification:



But this approach is prone to *aliasing*, which becomes even more prominent when the object is moving³.

³This is called *temporal aliasing*

Minification - Mipmapping

A Solution: *Mipmapping*

Minification - Mipmapping

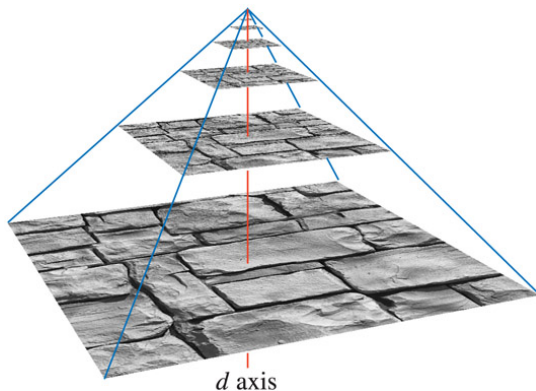
A Solution: *Mipmapping*

A Mipmap-Chain stores multiple resolutions of the original texture, reducing size to a quarter in each step:

Minification - Mipmapping

A Solution: *Mipmapping*

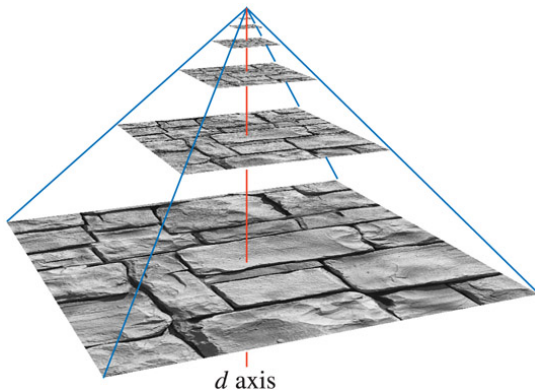
A Mipmap-Chain stores multiple resolutions of the original texture, reducing size to a quarter in each step:



Minification - Mipmapping

A Solution: *Mipmapping*

A Mipmap-Chain stores multiple resolutions of the original texture, reducing size to a quarter in each step:



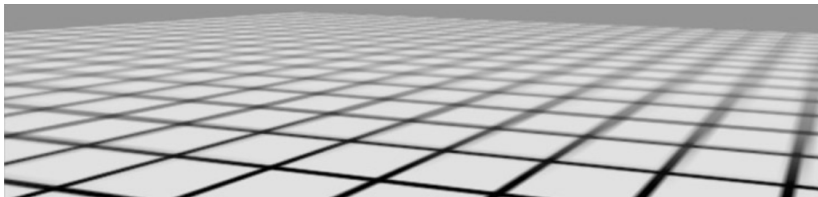
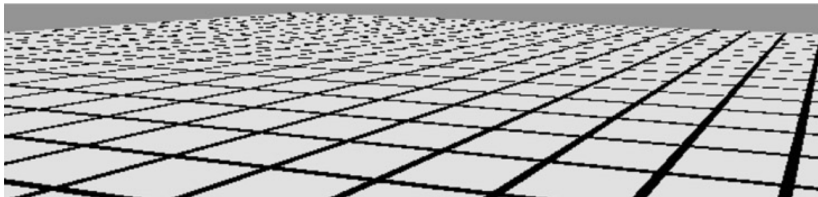
The mipmaps are either chosen with nearest neighbor or two adjacent levels are interpolated.

Minification - Mipmapping

With mip mapping the image is smoother and no longer prone to aliasing:

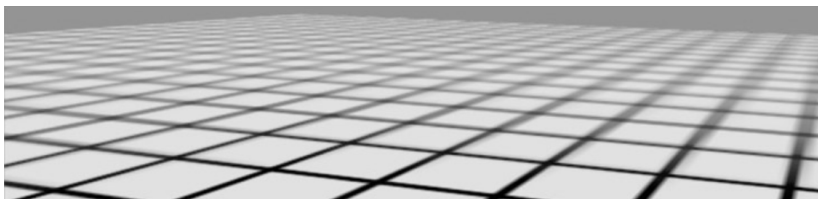
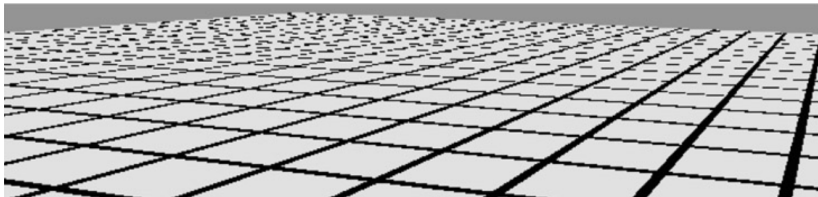
Minification - Mipmapping

With mip mapping the image is smoother and no longer prone to aliasing:



Minification - Mipmapping

With mip mapping the image is smoother and no longer prone to aliasing:



There is also the approach of *summed-area table (SAT)* and *anisotropic filtering*. It can be found together with the details of mipmapping in the related literature.

Rendering foliage with detailed geometry is often too expensive

Rendering foliage with detailed geometry is often too expensive but with Alpha mapping the illusion of detail geometry can be created:

Rendering foliage with detailed geometry is often too expensive but with Alpha mapping the illusion of detail geometry can be created:



Rendering foliage with detailed geometry is often too expensive but with Alpha mapping the illusion of detail geometry can be created:



This is created by rendering a 2D Texture and discarding pixels based on an alpha mask.

Rendering foliage with detailed geometry is often too expensive but with Alpha mapping the illusion of detail geometry can be created:



This is created by rendering a 2D Texture and discarding pixels based on an alpha mask.

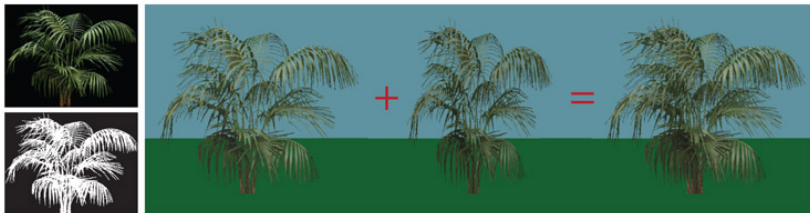
Rendering foliage with detailed geometry is often too expensive but with Alpha mapping the illusion of detail geometry can be created:



This is created by rendering a 2D Texture and discarding pixels based on an alpha mask.

```
if(texture.a  $\leq$  alphaThreshold) discard;
```

Rendering foliage with detailed geometry is often too expensive but with Alpha mapping the illusion of detail geometry can be created:



This is created by rendering a 2D Texture and discarding pixels based on an alpha mask.

```
if(texture.a  $\leq$  alphaThreshold) discard;
```

Usually two orthogonal planes are used for simple foliage.

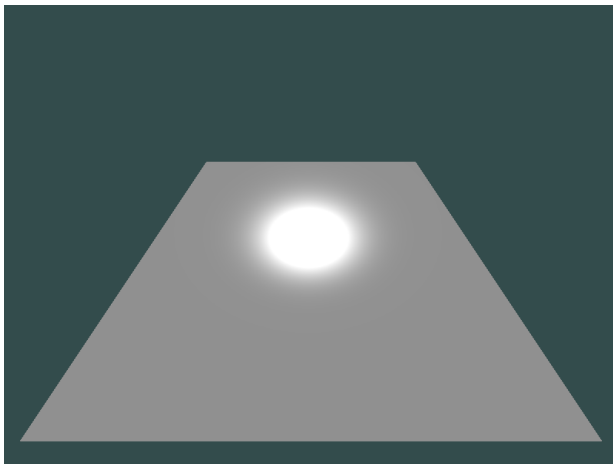
Alpha Mapping

Unfortunately the illusion breaks down when viewed from above:



Normal Mapping

In most shading models, the Normal of the surface is used to create lighting effects, such as specular highlights:

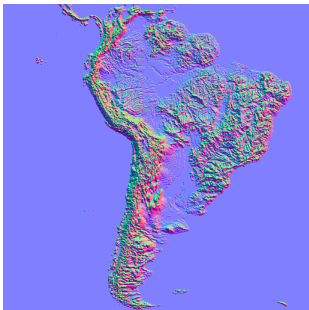


Normal Mapping

A normal map can be used give the lighting calculation more detail:

Normal Mapping

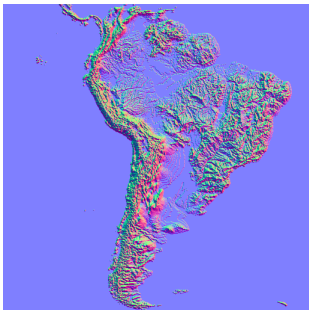
A normal map can be used give the lighting calculation more detail:



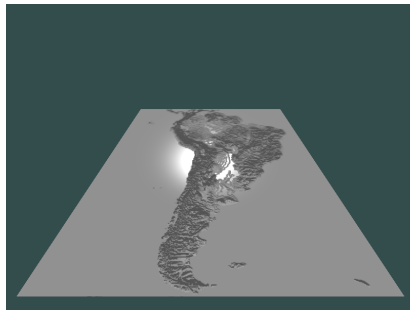
The normal texture encodes the surface normals, which have values of -1 to 1 in 0 to 1 color space.

Normal Mapping

A normal map can be used give the lighting calculation more detail:



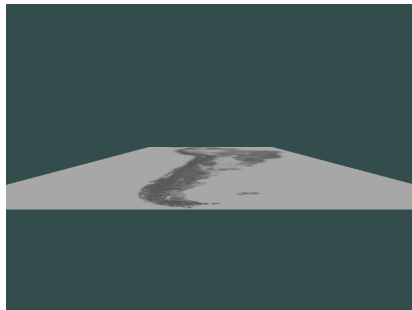
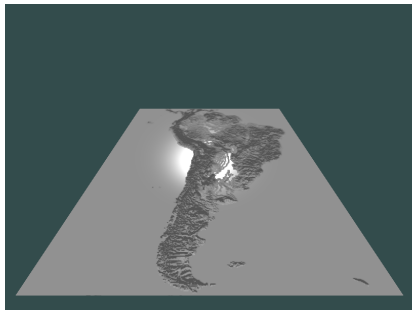
The normal texture encodes the surface normals, which have values of -1 to 1 in 0 to 1 color space.



Normal map applied to the lighting calculation of the plane for more depth. This creates an illusion of more geometry.

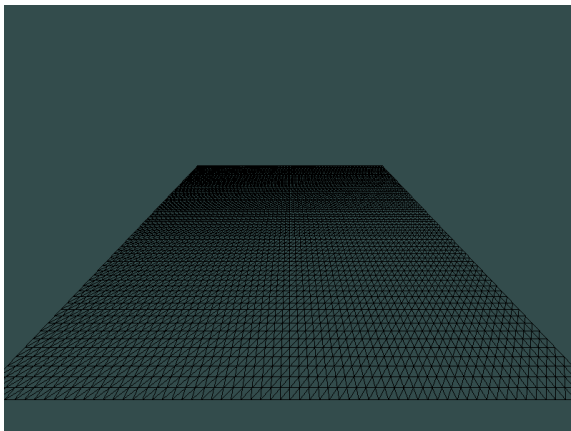
Normal Mapping

The effect of normals is heavily reduced when viewed from a low angle:



Displacement/Height Texture

With a *Height Texture* a tessellated mesh (or mesh with many vertices in a simple arrangement) can be displaced to create more detail:

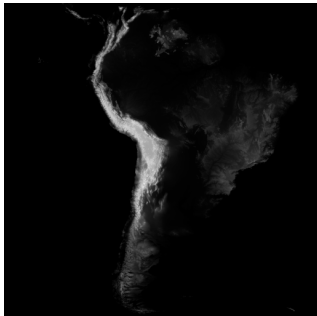


Displacement/Height Texture

With a *Height Texture* the vertices of a mesh are displaced along their surface normals:

Displacement/Height Texture

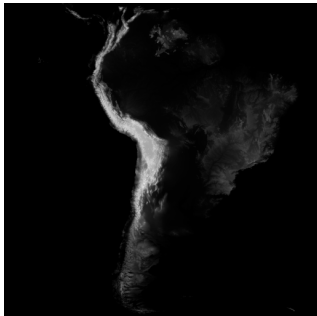
With a *Height Texture* the vertices of a mesh are displaced along their surface normals:



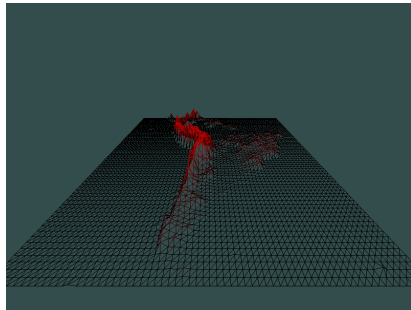
The height is store as one dimensional value in the height texture.

Displacement/Height Texture

With a *Height Texture* the vertices of a mesh are displaced along their surface normals:



The height is stored as one dimensional value in the height texture.



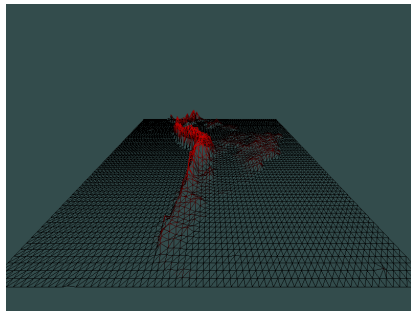
Displacement exaggerated and visualized in the red channel

Displacement/Height Texture

With a *Height Texture* the vertices of a mesh are displaced along their surface normals:



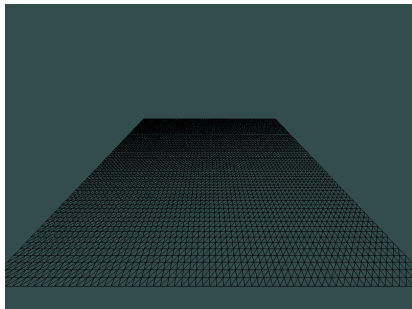
The height is stored as one dimensional value in the height texture.



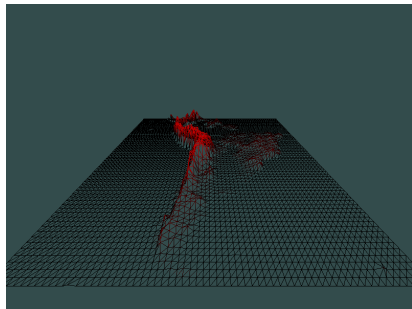
Displacement exaggerated and visualized in the red channel

This is often used in combination with tessellation or other runtime geometry generation approaches.

Displacement/Height Texture



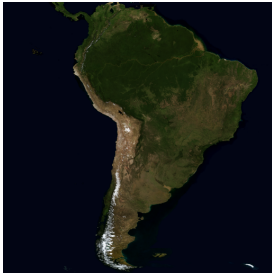
base mesh



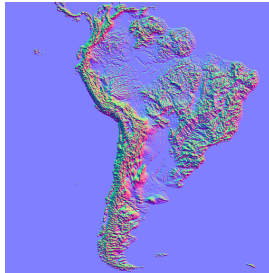
mesh with displacement applied

Combining Texture Approaches

By combining different textures in the shading the plane can be enhanced:



Color Texture



Normal Texture

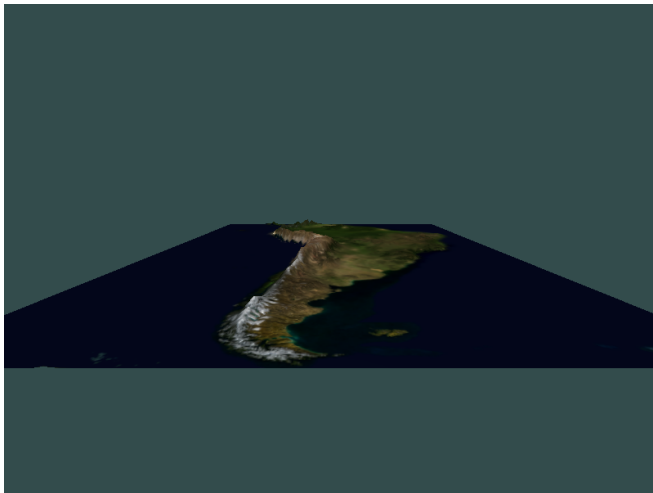


Height Texture

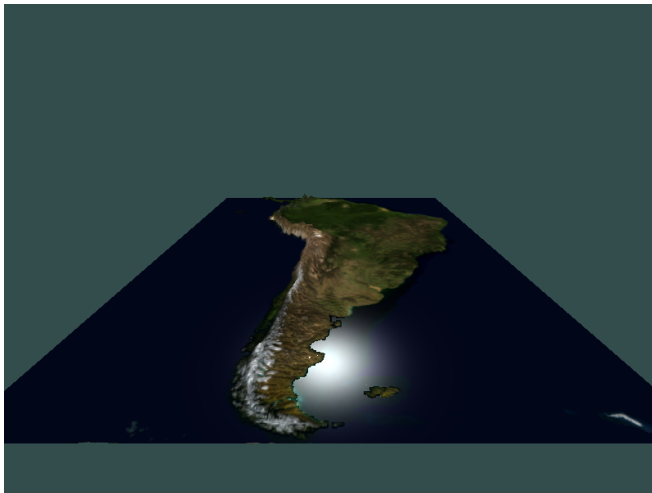
Combining Texture Approaches



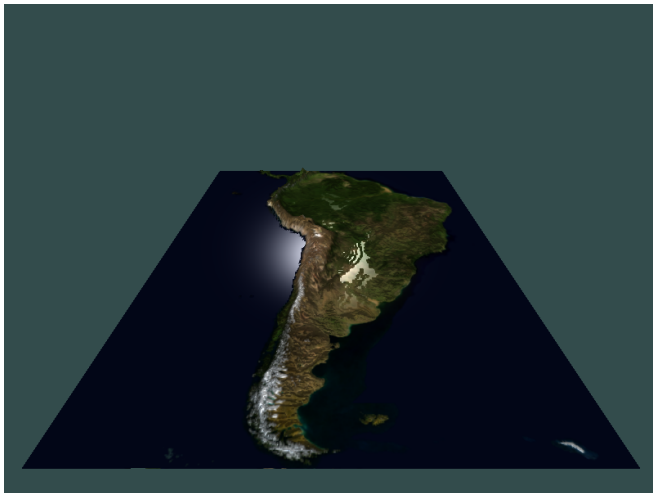
Combining Texture Approaches



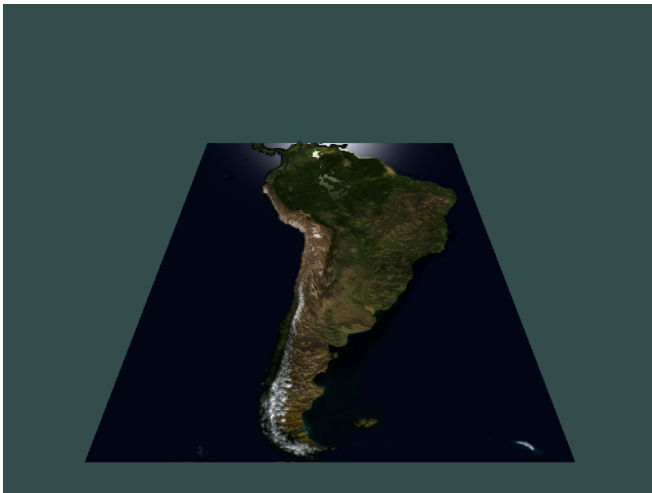
Combining Texture Approaches



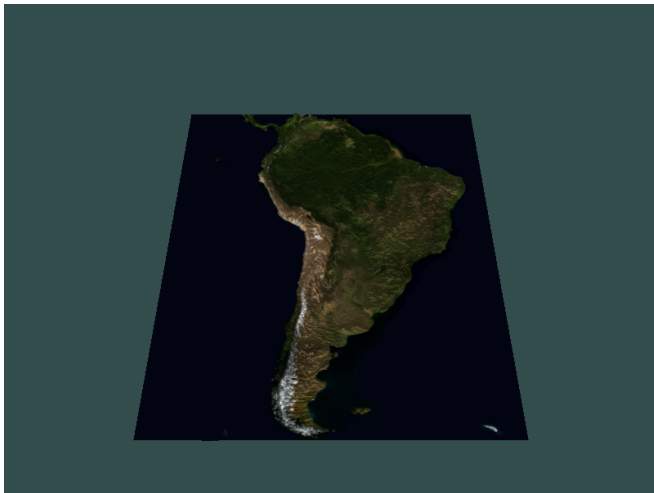
Combining Texture Approaches



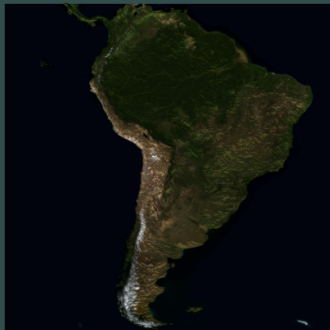
Combining Texture Approaches



Combining Texture Approaches



Combining Texture Approaches



Texture Resources:

- ▶ Visible Earth - Blue Marble

Model Resources:

- ▶ Duck (linked file contains a texture which may cause an error, use the file on moodle)
- ▶ Red Betta Fish by Elinor Quittner

Further Recommendation



I Tried Creating a Game Using Real-World Geographic Data by Sebastian Lague

Any Questions?

Average Person



Normal Person



Shader Error